Project Report

SUBJECT: COMPUTER GRAPHICS

Title

Finding Island: An OpenGL Simulation of Dynamic Scenarios

List of Group Members:

NAME	ID	CONTRIBUTION
PRETOM SHAHA	22-49709-3	SCENARIO-1
ISTIAK AHMMED SHIFAT	22-49319-3	SCENARIO-2
FAHIM ABRAR	22-49114-3	SCENARIO-3

- Member 1 – Core implementation of rendering engine, OpenGL functions, and animation logic.
- Member 2 – Development of scene designs (Village, Bridge, Island, Space).
- Member 3 – Integration of interactive elements and transitions between day/night and space modes.

Documentation, testing, debugging, and project report preparation.

Introduction

Computer graphics has become one of the most powerful tools for both research and practical applications in computer science, engineering, entertainment, and education. The rise of graphical simulations, ranging from simple 2D illustrations to immersive 3D virtual worlds, has allowed humans to visualize and interact with environments that would otherwise remain abstract. Among the many graphics libraries and frameworks available, OpenGL has remained one of the most widely used platforms because of its efficiency, cross-platform compatibility, and flexibility for both 2D and 3D rendering.

The project “Finding Island” was developed as a simulation-based exploration of real-world and imaginative scenes using OpenGL. The purpose of this project was to combine multiple animated environments, each representing unique aspects of nature, human-made structures, and even cosmic phenomena. The village scene demonstrates the blending of human life with nature, the bridge scene focuses on transportation and infrastructure, the island scene brings attention to solitary natural beauty, and the space

mode provides a glimpse into abstract futuristic visualization of the universe. These scenes were implemented using fundamental as well as advanced features of OpenGL including transformations, blending, time-based animation, and procedural geometry.

At its core, the project addresses one of the most common challenges faced in beginner to intermediate level computer graphics development: how to create modular yet cohesive animated systems where multiple independent objects (clouds, birds, cars, boats, planets, stars, debris, etc.) interact within a common timeline and rendering space. By integrating a modular design that encapsulates different visual elements into functions and structs, the project achieves both scalability and clarity. This structure allows developers and learners to not only run the simulation but also extend it easily with additional objects and behaviors.

In addition, the simulation integrates natural cycles such as day and night, with smooth visual transitions between the sun and moon, dynamic lighting colors for skies and waters, and different behaviors of elements (for example, birds only fly during the day, stars appear only at night). This attention to dynamic environmental conditions provides a more immersive experience and introduces students to the concept of environmental variables in graphical simulations.

The “Finding Island” project is not limited to education—it has potential applications in entertainment, as part of interactive storytelling, or as a foundation for simple 2D games. Furthermore, the inclusion of a space mode with futuristic features such as portals, crystals, auroras, and galaxies, illustrates how simulations can bridge the gap between realism and imagination. By demonstrating how code logic can generate artistic creativity, the project promotes interdisciplinary collaboration between computer science and digital art.

In total, this project demonstrates how OpenGL can be used not only to visualize but also to animate complex systems, while keeping performance in check. Through careful synchronization of moving entities, application of mathematical transformations, and modular programming practices, the “Finding Island” simulation serves as both a practical learning experience and an engaging showcase of graphical design.

Computer graphics has become one of the most powerful tools for both research and practical applications in computer science, engineering, entertainment, and education. The rise of graphical simulations, ranging from simple 2D illustrations to immersive 3D virtual worlds, has allowed humans to visualize and interact with environments that would otherwise remain abstract. Among the many graphics libraries and frameworks available, OpenGL has remained one of the most widely used platforms because of its

efficiency, cross-platform compatibility, and flexibility for both 2D and 3D rendering.

The project “Finding Island” was developed as a simulation-based exploration of real-world and imaginative scenes using OpenGL. The purpose of this project was to combine multiple animated environments, each representing unique aspects of nature, human-made structures, and even cosmic phenomena. The village scene demonstrates the blending of human life with nature, the bridge scene focuses on transportation and infrastructure, the island scene brings attention to solitary natural beauty, and the space mode provides a glimpse into abstract futuristic visualization of the universe. These scenes were implemented using fundamental as well as advanced features of OpenGL including transformations, blending, time-based animation, and procedural geometry.

At its core, the project addresses one of the most common challenges faced in beginner to intermediate level computer graphics development: how to create modular yet cohesive animated systems where multiple independent objects (clouds, birds, cars, boats, planets, stars, debris, etc.) interact within a common timeline and rendering space. By integrating a modular design that encapsulates different visual elements into functions and structs, the project achieves both scalability and clarity. This structure allows developers and learners to not only run the simulation but also extend it easily with additional objects and behaviors.

In addition, the simulation integrates natural cycles such as day and night, with smooth visual transitions between the sun and moon, dynamic lighting colors for skies and waters, and different behaviors of elements (for example, birds only fly during the day, stars appear only at night). This attention to dynamic environmental conditions provides a more immersive experience and introduces students to the concept of environmental variables in graphical simulations.

The “Finding Island” project is not limited to education—it has potential applications in entertainment, as part of interactive storytelling, or as a foundation for simple 2D games. Furthermore, the inclusion of a space mode with futuristic features such as portals, crystals, auroras, and galaxies, illustrates how simulations can bridge the gap between realism and imagination. By demonstrating how code logic can generate artistic creativity, the project promotes interdisciplinary collaboration between computer science and digital art.

In total, this project demonstrates how OpenGL can be used not only to visualize but also to animate complex systems, while keeping performance in check. Through careful synchronization of moving entities, application of mathematical transformations, and modular programming practices, the “Finding Island” simulation serves as both a

practical learning experience and an engaging showcase of graphical design.

Computer graphics has become one of the most powerful tools for both research and practical applications in computer science, engineering, entertainment, and education. The rise of graphical simulations, ranging from simple 2D illustrations to immersive 3D virtual worlds, has allowed humans to visualize and interact with environments that would otherwise remain abstract. Among the many graphics libraries and frameworks available, OpenGL has remained one of the most widely used platforms because of its efficiency, cross-platform compatibility, and flexibility for both 2D and 3D rendering.

The project “Finding Island” was developed as a simulation-based exploration of real-world and imaginative scenes using OpenGL. The purpose of this project was to combine multiple animated environments, each representing unique aspects of nature, human-made structures, and even cosmic phenomena. The village scene demonstrates the blending of human life with nature, the bridge scene focuses on transportation and infrastructure, the island scene brings attention to solitary natural beauty, and the space mode provides a glimpse into abstract futuristic visualization of the universe. These scenes were implemented using fundamental as well as advanced features of OpenGL including transformations, blending, time-based animation, and procedural geometry.

At its core, the project addresses one of the most common challenges faced in beginner to intermediate level computer graphics development: how to create modular yet cohesive animated systems where multiple independent objects (clouds, birds, cars, boats, planets, stars, debris, etc.) interact within a common timeline and rendering space. By integrating a modular design that encapsulates different visual elements into functions and structs, the project achieves both scalability and clarity. This structure allows developers and learners to not only run the simulation but also extend it easily with additional objects and behaviors.

In addition, the simulation integrates natural cycles such as day and night, with smooth visual transitions between the sun and moon, dynamic lighting colors for skies and waters, and different behaviors of elements (for example, birds only fly during the day, stars appear only at night). This attention to dynamic environmental conditions provides a more immersive experience and introduces students to the concept of environmental variables in graphical simulations.

The “Finding Island” project is not limited to education—it has potential applications in entertainment, as part of interactive storytelling, or as a foundation for simple 2D games. Furthermore, the inclusion of a space mode with futuristic features such as portals, crystals, auroras, and galaxies, illustrates how simulations can bridge the gap between

realism and imagination. By demonstrating how code logic can generate artistic creativity, the project promotes interdisciplinary collaboration between computer science and digital art.

In total, this project demonstrates how OpenGL can be used not only to visualize but also to animate complex systems, while keeping performance in check. Through careful synchronization of moving entities, application of mathematical transformations, and modular programming practices, the “Finding Island” simulation serves as both a practical learning experience and an engaging showcase of graphical design.

Computer graphics has become one of the most powerful tools for both research and practical applications in computer science, engineering, entertainment, and education. The rise of graphical simulations, ranging from simple 2D illustrations to immersive 3D virtual worlds, has allowed humans to visualize and interact with environments that would otherwise remain abstract. Among the many graphics libraries and frameworks available, OpenGL has remained one of the most widely used platforms because of its efficiency, cross-platform compatibility, and flexibility for both 2D and 3D rendering.

The project “Finding Island” was developed as a simulation-based exploration of real-world and imaginative scenes using OpenGL. The purpose of this project was to combine multiple animated environments, each representing unique aspects of nature, human-made structures, and even cosmic phenomena. The village scene demonstrates the blending of human life with nature, the bridge scene focuses on transportation and infrastructure, the island scene brings attention to solitary natural beauty, and the space mode provides a glimpse into abstract futuristic visualization of the universe. These scenes were implemented using fundamental as well as advanced features of OpenGL including transformations, blending, time-based animation, and procedural geometry.

At its core, the project addresses one of the most common challenges faced in beginner to intermediate level computer graphics development: how to create modular yet cohesive animated systems where multiple independent objects (clouds, birds, cars, boats, planets, stars, debris, etc.) interact within a common timeline and rendering space. By integrating a modular design that encapsulates different visual elements into functions and structs, the project achieves both scalability and clarity. This structure allows developers and learners to not only run the simulation but also extend it easily with additional objects and behaviors.

In addition, the simulation integrates natural cycles such as day and night, with smooth visual transitions between the sun and moon, dynamic lighting colors for skies and waters, and different behaviors of elements (for example, birds only fly during the day,

stars appear only at night). This attention to dynamic environmental conditions provides a more immersive experience and introduces students to the concept of environmental variables in graphical simulations.

The “Finding Island” project is not limited to education—it has potential applications in entertainment, as part of interactive storytelling, or as a foundation for simple 2D games. Furthermore, the inclusion of a space mode with futuristic features such as portals, crystals, auroras, and galaxies, illustrates how simulations can bridge the gap between realism and imagination. By demonstrating how code logic can generate artistic creativity, the project promotes interdisciplinary collaboration between computer science and digital art.

In total, this project demonstrates how OpenGL can be used not only to visualize but also to animate complex systems, while keeping performance in check. Through careful synchronization of moving entities, application of mathematical transformations, and modular programming practices, the “Finding Island” simulation serves as both a practical learning experience and an engaging showcase of graphical design.

Computer graphics has become one of the most powerful tools for both research and practical applications in computer science, engineering, entertainment, and education. The rise of graphical simulations, ranging from simple 2D illustrations to immersive 3D virtual worlds, has allowed humans to visualize and interact with environments that would otherwise remain abstract. Among the many graphics libraries and frameworks available, OpenGL has remained one of the most widely used platforms because of its efficiency, cross-platform compatibility, and flexibility for both 2D and 3D rendering.

The project “Finding Island” was developed as a simulation-based exploration of real-world and imaginative scenes using OpenGL. The purpose of this project was to combine multiple animated environments, each representing unique aspects of nature, human-made structures, and even cosmic phenomena. The village scene demonstrates the blending of human life with nature, the bridge scene focuses on transportation and infrastructure, the island scene brings attention to solitary natural beauty, and the space mode provides a glimpse into abstract futuristic visualization of the universe. These scenes were implemented using fundamental as well as advanced features of OpenGL including transformations, blending, time-based animation, and procedural geometry.

At its core, the project addresses one of the most common challenges faced in beginner to intermediate level computer graphics development: how to create modular yet cohesive animated systems where multiple independent objects (clouds, birds, cars, boats, planets, stars, debris, etc.) interact within a common timeline and rendering space. By integrating

a modular design that encapsulates different visual elements into functions and structs, the project achieves both scalability and clarity. This structure allows developers and learners to not only run the simulation but also extend it easily with additional objects and behaviors.

In addition, the simulation integrates natural cycles such as day and night, with smooth visual transitions between the sun and moon, dynamic lighting colors for skies and waters, and different behaviors of elements (for example, birds only fly during the day, stars appear only at night). This attention to dynamic environmental conditions provides a more immersive experience and introduces students to the concept of environmental variables in graphical simulations.

The “Finding Island” project is not limited to education—it has potential applications in entertainment, as part of interactive storytelling, or as a foundation for simple 2D games. Furthermore, the inclusion of a space mode with futuristic features such as portals, crystals, auroras, and galaxies, illustrates how simulations can bridge the gap between realism and imagination. By demonstrating how code logic can generate artistic creativity, the project promotes interdisciplinary collaboration between computer science and digital art.

In total, this project demonstrates how OpenGL can be used not only to visualize but also to animate complex systems, while keeping performance in check. Through careful synchronization of moving entities, application of mathematical transformations, and modular programming practices, the “Finding Island” simulation serves as both a practical learning experience and an engaging showcase of graphical design.

Background of the Problem

The motivation behind this project arises from the need to teach and practice fundamental graphics programming concepts in a structured, scenario-based approach. Computer graphics students often begin by learning how to draw simple shapes, lines, and transformations. However, the real challenge begins when these individual drawings must interact in a cohesive animation that responds to environmental changes. “Finding Island” was developed to address these challenges by combining both static and dynamic graphical components into a single simulation.

One of the primary background problems faced in real-time graphics development is the synchronization of moving elements. For example, cars on a road must appear to move at realistic speeds relative to boats in the water, while birds and airplanes must follow

independent flight paths across the sky. These interactions require a global sense of time and consistent update functions that allow objects to move harmoniously without conflict or visual glitches. The code implements this through a global time counter and update functions that increment object positions based on specific velocities.

Another important issue addressed by the project is the transition between scenarios and environments. Many beginner projects are limited to static scenes or require the program to restart to display different environments. In this simulation, transitions are handled smoothly, allowing the user to experience multiple scenarios—village, bridge, island, and space—within the same running program. This provides flexibility and expands the scope of the learning experience.

The background problem also includes visual realism and engagement. Without proper rendering of skies, water, and lighting, graphical simulations appear flat and uninteresting. “Finding Island” solves this by including enhanced rendering options such as gradients in the sky, reflective surfaces on the water, glowing effects for stars and shooting stars, and transparency effects for auroras. These additions not only improve realism but also expose learners to advanced OpenGL features such as blending and alpha transparency.

In the modern context, another motivating factor is that computer graphics is no longer limited to desktop environments. With the rise of augmented reality (AR) and virtual reality (VR), as well as mobile applications, the ability to create lightweight yet dynamic graphical simulations is critical. “Finding Island” serves as a stepping stone towards this direction, giving developers the skills needed to later integrate shaders, textures, and even 3D objects into their work.

The motivation behind this project arises from the need to teach and practice fundamental graphics programming concepts in a structured, scenario-based approach. Computer graphics students often begin by learning how to draw simple shapes, lines, and transformations. However, the real challenge begins when these individual drawings must interact in a cohesive animation that responds to environmental changes. “Finding Island” was developed to address these challenges by combining both static and dynamic graphical components into a single simulation.

One of the primary background problems faced in real-time graphics development is the synchronization of moving elements. For example, cars on a road must appear to move at realistic speeds relative to boats in the water, while birds and airplanes must follow independent flight paths across the sky. These interactions require a global sense of time and consistent update functions that allow objects to move harmoniously without conflict

or visual glitches. The code implements this through a global time counter and update functions that increment object positions based on specific velocities.

Another important issue addressed by the project is the transition between scenarios and environments. Many beginner projects are limited to static scenes or require the program to restart to display different environments. In this simulation, transitions are handled smoothly, allowing the user to experience multiple scenarios—village, bridge, island, and space—within the same running program. This provides flexibility and expands the scope of the learning experience.

The background problem also includes visual realism and engagement. Without proper rendering of skies, water, and lighting, graphical simulations appear flat and uninteresting. “Finding Island” solves this by including enhanced rendering options such as gradients in the sky, reflective surfaces on the water, glowing effects for stars and shooting stars, and transparency effects for auroras. These additions not only improve realism but also expose learners to advanced OpenGL features such as blending and alpha transparency.

In the modern context, another motivating factor is that computer graphics is no longer limited to desktop environments. With the rise of augmented reality (AR) and virtual reality (VR), as well as mobile applications, the ability to create lightweight yet dynamic graphical simulations is critical. “Finding Island” serves as a stepping stone towards this direction, giving developers the skills needed to later integrate shaders, textures, and even 3D objects into their work.

The motivation behind this project arises from the need to teach and practice fundamental graphics programming concepts in a structured, scenario-based approach. Computer graphics students often begin by learning how to draw simple shapes, lines, and transformations. However, the real challenge begins when these individual drawings must interact in a cohesive animation that responds to environmental changes. “Finding Island” was developed to address these challenges by combining both static and dynamic graphical components into a single simulation.

One of the primary background problems faced in real-time graphics development is the synchronization of moving elements. For example, cars on a road must appear to move at realistic speeds relative to boats in the water, while birds and airplanes must follow independent flight paths across the sky. These interactions require a global sense of time and consistent update functions that allow objects to move harmoniously without conflict or visual glitches. The code implements this through a global time counter and update functions that increment object positions based on specific velocities.

Another important issue addressed by the project is the transition between scenarios and environments. Many beginner projects are limited to static scenes or require the program to restart to display different environments. In this simulation, transitions are handled smoothly, allowing the user to experience multiple scenarios—village, bridge, island, and space—within the same running program. This provides flexibility and expands the scope of the learning experience.

The background problem also includes visual realism and engagement. Without proper rendering of skies, water, and lighting, graphical simulations appear flat and uninteresting. “Finding Island” solves this by including enhanced rendering options such as gradients in the sky, reflective surfaces on the water, glowing effects for stars and shooting stars, and transparency effects for auroras. These additions not only improve realism but also expose learners to advanced OpenGL features such as blending and alpha transparency.

In the modern context, another motivating factor is that computer graphics is no longer limited to desktop environments. With the rise of augmented reality (AR) and virtual reality (VR), as well as mobile applications, the ability to create lightweight yet dynamic graphical simulations is critical. “Finding Island” serves as a stepping stone towards this direction, giving developers the skills needed to later integrate shaders, textures, and even 3D objects into their work.

The motivation behind this project arises from the need to teach and practice fundamental graphics programming concepts in a structured, scenario-based approach. Computer graphics students often begin by learning how to draw simple shapes, lines, and transformations. However, the real challenge begins when these individual drawings must interact in a cohesive animation that responds to environmental changes. “Finding Island” was developed to address these challenges by combining both static and dynamic graphical components into a single simulation.

One of the primary background problems faced in real-time graphics development is the synchronization of moving elements. For example, cars on a road must appear to move at realistic speeds relative to boats in the water, while birds and airplanes must follow independent flight paths across the sky. These interactions require a global sense of time and consistent update functions that allow objects to move harmoniously without conflict or visual glitches. The code implements this through a global time counter and update functions that increment object positions based on specific velocities.

Another important issue addressed by the project is the transition between scenarios and

environments. Many beginner projects are limited to static scenes or require the program to restart to display different environments. In this simulation, transitions are handled smoothly, allowing the user to experience multiple scenarios—village, bridge, island, and space—within the same running program. This provides flexibility and expands the scope of the learning experience.

The background problem also includes visual realism and engagement. Without proper rendering of skies, water, and lighting, graphical simulations appear flat and uninteresting. “Finding Island” solves this by including enhanced rendering options such as gradients in the sky, reflective surfaces on the water, glowing effects for stars and shooting stars, and transparency effects for auroras. These additions not only improve realism but also expose learners to advanced OpenGL features such as blending and alpha transparency.

In the modern context, another motivating factor is that computer graphics is no longer limited to desktop environments. With the rise of augmented reality (AR) and virtual reality (VR), as well as mobile applications, the ability to create lightweight yet dynamic graphical simulations is critical. “Finding Island” serves as a stepping stone towards this direction, giving developers the skills needed to later integrate shaders, textures, and even 3D objects into their work.

The motivation behind this project arises from the need to teach and practice fundamental graphics programming concepts in a structured, scenario-based approach. Computer graphics students often begin by learning how to draw simple shapes, lines, and transformations. However, the real challenge begins when these individual drawings must interact in a cohesive animation that responds to environmental changes. “Finding Island” was developed to address these challenges by combining both static and dynamic graphical components into a single simulation.

One of the primary background problems faced in real-time graphics development is the synchronization of moving elements. For example, cars on a road must appear to move at realistic speeds relative to boats in the water, while birds and airplanes must follow independent flight paths across the sky. These interactions require a global sense of time and consistent update functions that allow objects to move harmoniously without conflict or visual glitches. The code implements this through a global time counter and update functions that increment object positions based on specific velocities.

Another important issue addressed by the project is the transition between scenarios and environments. Many beginner projects are limited to static scenes or require the program to restart to display different environments. In this simulation, transitions are handled

smoothly, allowing the user to experience multiple scenarios—village, bridge, island, and space—within the same running program. This provides flexibility and expands the scope of the learning experience.

The background problem also includes visual realism and engagement. Without proper rendering of skies, water, and lighting, graphical simulations appear flat and uninteresting. “Finding Island” solves this by including enhanced rendering options such as gradients in the sky, reflective surfaces on the water, glowing effects for stars and shooting stars, and transparency effects for auroras. These additions not only improve realism but also expose learners to advanced OpenGL features such as blending and alpha transparency.

In the modern context, another motivating factor is that computer graphics is no longer limited to desktop environments. With the rise of augmented reality (AR) and virtual reality (VR), as well as mobile applications, the ability to create lightweight yet dynamic graphical simulations is critical. “Finding Island” serves as a stepping stone towards this direction, giving developers the skills needed to later integrate shaders, textures, and even 3D objects into their work.

Methods Used

The methods used in the development of “Finding Island” are primarily based on OpenGL’s fixed-function pipeline and 2D rendering techniques. The simulation is structured around modular functions, each responsible for rendering a specific part of the scene. For example, `drawSky()` handles the sky gradient and stars, `drawWaterAndWaves()` simulates moving water surfaces, and `drawCarsAndAirplane()` handles ground transportation. This modularity is key to making the system extensible and easy to maintain.

The first step in the methodology was setting up the coordinate system using `gluOrtho2D`. This established a 2D orthographic projection where the x-axis represents horizontal pixel positions and the y-axis represents vertical pixel positions. With this setup, it became straightforward to position objects at precise screen coordinates. The window dimensions were fixed to 1000x700, ensuring consistency in rendering.

Time-based animation was achieved using an `update()` function that increments a `globalTime` variable. Every frame, object positions such as the sun, clouds, birds, and boats are updated based on their velocities. Trigonometric functions (`sinf`, `cosf`) are used extensively to generate natural movements such as circular or elliptical paths (for

planets, craters, and waves).

Structs such as ShootingStar, Planet, Aurora, and SpaceDebris were defined to encapsulate properties of different dynamic objects. Each struct contains attributes such as position, velocity, radius, angle, color, and activity status. Update functions modify these attributes while draw functions render the objects to the screen. This object-based modeling ensures clarity and prevents redundancy in the code.

Transparency and blending were achieved using `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. These settings enabled effects such as glowing trails for shooting stars, semi-transparent auroras, and soft water reflections. By using layered rendering (multiple draw passes with varying alpha values), a sense of depth and realism was introduced even in a purely 2D simulation.

The scenarios were implemented as independent draw functions (`drawScenario0`, `drawBridgeScene`, `drawIslandScene`). A scenario switch variable allows the program to change between them without restarting. Enhanced mode and space mode are toggles that modify the rendering pipeline, activating additional features like nebula backgrounds, galaxies, and portals.

Overall, the methods used combined mathematical modeling, OpenGL rendering functions, modular programming, and object-oriented principles to build a comprehensive simulation.

The methods used in the development of “Finding Island” are primarily based on OpenGL’s fixed-function pipeline and 2D rendering techniques. The simulation is structured around modular functions, each responsible for rendering a specific part of the scene. For example, `drawSky()` handles the sky gradient and stars, `drawWaterAndWaves()` simulates moving water surfaces, and `drawCarsAndAirplane()` handles ground transportation. This modularity is key to making the system extensible and easy to maintain.

The first step in the methodology was setting up the coordinate system using `gluOrtho2D`. This established a 2D orthographic projection where the x-axis represents horizontal pixel positions and the y-axis represents vertical pixel positions. With this setup, it became straightforward to position objects at precise screen coordinates. The window dimensions were fixed to 1000x700, ensuring consistency in rendering.

Time-based animation was achieved using an `update()` function that increments a

`globalTime` variable. Every frame, object positions such as the sun, clouds, birds, and boats are updated based on their velocities. Trigonometric functions (`sinf`, `cosf`) are used extensively to generate natural movements such as circular or elliptical paths (for planets, craters, and waves).

Structs such as ShootingStar, Planet, Aurora, and SpaceDebris were defined to encapsulate properties of different dynamic objects. Each struct contains attributes such as position, velocity, radius, angle, color, and activity status. Update functions modify these attributes while draw functions render the objects to the screen. This object-based modeling ensures clarity and prevents redundancy in the code.

Transparency and blending were achieved using `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. These settings enabled effects such as glowing trails for shooting stars, semi-transparent auroras, and soft water reflections. By using layered rendering (multiple draw passes with varying alpha values), a sense of depth and realism was introduced even in a purely 2D simulation.

The scenarios were implemented as independent draw functions (`drawScenario0`, `drawBridgeScene`, `drawIslandScene`). A scenario switch variable allows the program to change between them without restarting. Enhanced mode and space mode are toggles that modify the rendering pipeline, activating additional features like nebula backgrounds, galaxies, and portals.

Overall, the methods used combined mathematical modeling, OpenGL rendering functions, modular programming, and object-oriented principles to build a comprehensive simulation.

The methods used in the development of “Finding Island” are primarily based on OpenGL’s fixed-function pipeline and 2D rendering techniques. The simulation is structured around modular functions, each responsible for rendering a specific part of the scene. For example, `drawSky()` handles the sky gradient and stars, `drawWaterAndWaves()` simulates moving water surfaces, and `drawCarsAndAirplane()` handles ground transportation. This modularity is key to making the system extensible and easy to maintain.

The first step in the methodology was setting up the coordinate system using `gluOrtho2D`. This established a 2D orthographic projection where the x-axis represents horizontal pixel positions and the y-axis represents vertical pixel positions. With this setup, it became straightforward to position objects at precise screen coordinates. The

window dimensions were fixed to 1000x700, ensuring consistency in rendering.

Time-based animation was achieved using an `update()` function that increments a `globalTime` variable. Every frame, object positions such as the sun, clouds, birds, and boats are updated based on their velocities. Trigonometric functions (`sinf`, `cosf`) are used extensively to generate natural movements such as circular or elliptical paths (for planets, craters, and waves).

Structs such as ShootingStar, Planet, Aurora, and SpaceDebris were defined to encapsulate properties of different dynamic objects. Each struct contains attributes such as position, velocity, radius, angle, color, and activity status. Update functions modify these attributes while draw functions render the objects to the screen. This object-based modeling ensures clarity and prevents redundancy in the code.

Transparency and blending were achieved using `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. These settings enabled effects such as glowing trails for shooting stars, semi-transparent auroras, and soft water reflections. By using layered rendering (multiple draw passes with varying alpha values), a sense of depth and realism was introduced even in a purely 2D simulation.

The scenarios were implemented as independent draw functions (`drawScenario0`, `drawBridgeScene`, `drawIslandScene`). A scenario switch variable allows the program to change between them without restarting. Enhanced mode and space mode are toggles that modify the rendering pipeline, activating additional features like nebula backgrounds, galaxies, and portals.

Overall, the methods used combined mathematical modeling, OpenGL rendering functions, modular programming, and object-oriented principles to build a comprehensive simulation.

The methods used in the development of “Finding Island” are primarily based on OpenGL’s fixed-function pipeline and 2D rendering techniques. The simulation is structured around modular functions, each responsible for rendering a specific part of the scene. For example, `drawSky()` handles the sky gradient and stars, `drawWaterAndWaves()` simulates moving water surfaces, and `drawCarsAndAirplane()` handles ground transportation. This modularity is key to making the system extensible and easy to maintain.

The first step in the methodology was setting up the coordinate system using

`gluOrtho2D`. This established a 2D orthographic projection where the x-axis represents horizontal pixel positions and the y-axis represents vertical pixel positions. With this setup, it became straightforward to position objects at precise screen coordinates. The window dimensions were fixed to 1000x700, ensuring consistency in rendering.

Time-based animation was achieved using an `update()` function that increments a `globalTime` variable. Every frame, object positions such as the sun, clouds, birds, and boats are updated based on their velocities. Trigonometric functions (`sinf`, `cosf`) are used extensively to generate natural movements such as circular or elliptical paths (for planets, craters, and waves).

Structs such as ShootingStar, Planet, Aurora, and SpaceDebris were defined to encapsulate properties of different dynamic objects. Each struct contains attributes such as position, velocity, radius, angle, color, and activity status. Update functions modify these attributes while draw functions render the objects to the screen. This object-based modeling ensures clarity and prevents redundancy in the code.

Transparency and blending were achieved using `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. These settings enabled effects such as glowing trails for shooting stars, semi-transparent auroras, and soft water reflections. By using layered rendering (multiple draw passes with varying alpha values), a sense of depth and realism was introduced even in a purely 2D simulation.

The scenarios were implemented as independent draw functions (`drawScenario0`, `drawBridgeScene`, `drawIslandScene`). A scenario switch variable allows the program to change between them without restarting. Enhanced mode and space mode are toggles that modify the rendering pipeline, activating additional features like nebula backgrounds, galaxies, and portals.

Overall, the methods used combined mathematical modeling, OpenGL rendering functions, modular programming, and object-oriented principles to build a comprehensive simulation.

The methods used in the development of “Finding Island” are primarily based on OpenGL’s fixed-function pipeline and 2D rendering techniques. The simulation is structured around modular functions, each responsible for rendering a specific part of the scene. For example, `drawSky()` handles the sky gradient and stars, `drawWaterAndWaves()` simulates moving water surfaces, and `drawCarsAndAirplane()` handles ground transportation. This modularity is key to

making the system extensible and easy to maintain.

The first step in the methodology was setting up the coordinate system using `gluOrtho2D`. This established a 2D orthographic projection where the x-axis represents horizontal pixel positions and the y-axis represents vertical pixel positions. With this setup, it became straightforward to position objects at precise screen coordinates. The window dimensions were fixed to 1000x700, ensuring consistency in rendering.

Time-based animation was achieved using an `update()` function that increments a `globalTime` variable. Every frame, object positions such as the sun, clouds, birds, and boats are updated based on their velocities. Trigonometric functions (`sinf`, `cosf`) are used extensively to generate natural movements such as circular or elliptical paths (for planets, craters, and waves).

Structs such as ShootingStar, Planet, Aurora, and SpaceDebris were defined to encapsulate properties of different dynamic objects. Each struct contains attributes such as position, velocity, radius, angle, color, and activity status. Update functions modify these attributes while draw functions render the objects to the screen. This object-based modeling ensures clarity and prevents redundancy in the code.

Transparency and blending were achieved using `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. These settings enabled effects such as glowing trails for shooting stars, semi-transparent auroras, and soft water reflections. By using layered rendering (multiple draw passes with varying alpha values), a sense of depth and realism was introduced even in a purely 2D simulation.

The scenarios were implemented as independent draw functions (`drawScenario0`, `drawBridgeScene`, `drawIslandScene`). A scenario switch variable allows the program to change between them without restarting. Enhanced mode and space mode are toggles that modify the rendering pipeline, activating additional features like nebula backgrounds, galaxies, and portals.

Overall, the methods used combined mathematical modeling, OpenGL rendering functions, modular programming, and object-oriented principles to build a comprehensive simulation.

Feature Set

The “Finding Island” simulation offers a wide range of features that demonstrate the versatility of OpenGL.

1. Day and Night Cycle – The system transitions between day and night using different sky gradients, replacing the sun with the moon, and enabling/disabling stars and birds accordingly.
2. Animated Clouds and Birds – Clouds drift across the sky at different speeds, while birds flap their wings and move along parabolic paths, active only during the day.
3. Water and Waves – The water body is animated with sine waves that simulate realistic wave motion. Reflections of the moon are also added at night.
4. Boats – A fishing boat and cargo boat move across the water, with bobbing effects to simulate buoyancy. Human figures are also rendered on the boats to enhance realism.
5. Cars and Roads – Cars of different colors and sizes move along the road, while an airplane flies across the sky, adding dynamism to the man-made scene.
6. Village and House – A small village environment with houses and trees is constructed to ground the scene in a natural rural setting.
7. Bridge Scene – A separate scenario features a bridge over water, complete with lighting effects, structural supports, and traffic.
8. Island Scene – The island features palm trees, landmasses, and greenery during normal mode, while in space mode it transforms into a futuristic scene with crystals and portals.
9. Space Mode – Includes planets with orbital motion, auroras, shooting stars, galaxies, and space debris. These features highlight OpenGL’s ability to simulate imaginative phenomena.
10. Enhanced Mode – Adds additional graphical polish such as glowing star effects, more complex gradients, and additional transparency layers.

The combination of these features ensures that the simulation is not only educational but also visually engaging. Each feature was carefully integrated to demonstrate different aspects of OpenGL rendering.

The “Finding Island” simulation offers a wide range of features that demonstrate the versatility of OpenGL.

1. Day and Night Cycle – The system transitions between day and night using different sky gradients, replacing the sun with the moon, and enabling/disabling stars and birds accordingly.
2. Animated Clouds and Birds – Clouds drift across the sky at different speeds, while birds flap their wings and move along parabolic paths, active only during the day.

3. Water and Waves – The water body is animated with sine waves that simulate realistic wave motion. Reflections of the moon are also added at night.
4. Boats – A fishing boat and cargo boat move across the water, with bobbing effects to simulate buoyancy. Human figures are also rendered on the boats to enhance realism.
5. Cars and Roads – Cars of different colors and sizes move along the road, while an airplane flies across the sky, adding dynamism to the man-made scene.
6. Village and House – A small village environment with houses and trees is constructed to ground the scene in a natural rural setting.
7. Bridge Scene – A separate scenario features a bridge over water, complete with lighting effects, structural supports, and traffic.
8. Island Scene – The island features palm trees, landmasses, and greenery during normal mode, while in space mode it transforms into a futuristic scene with crystals and portals.
9. Space Mode – Includes planets with orbital motion, auroras, shooting stars, galaxies, and space debris. These features highlight OpenGL's ability to simulate imaginative phenomena.
10. Enhanced Mode – Adds additional graphical polish such as glowing star effects, more complex gradients, and additional transparency layers.

The combination of these features ensures that the simulation is not only educational but also visually engaging. Each feature was carefully integrated to demonstrate different aspects of OpenGL rendering.

The "Finding Island" simulation offers a wide range of features that demonstrate the versatility of OpenGL.

1. Day and Night Cycle – The system transitions between day and night using different sky gradients, replacing the sun with the moon, and enabling/disabling stars and birds accordingly.
2. Animated Clouds and Birds – Clouds drift across the sky at different speeds, while birds flap their wings and move along parabolic paths, active only during the day.
3. Water and Waves – The water body is animated with sine waves that simulate realistic wave motion. Reflections of the moon are also added at night.
4. Boats – A fishing boat and cargo boat move across the water, with bobbing effects to simulate buoyancy. Human figures are also rendered on the boats to enhance realism.
5. Cars and Roads – Cars of different colors and sizes move along the road, while an airplane flies across the sky, adding dynamism to the man-made scene.
6. Village and House – A small village environment with houses and trees is constructed to ground the scene in a natural rural setting.
7. Bridge Scene – A separate scenario features a bridge over water, complete with lighting effects, structural supports, and traffic.

8. Island Scene – The island features palm trees, landmasses, and greenery during normal mode, while in space mode it transforms into a futuristic scene with crystals and portals.
9. Space Mode – Includes planets with orbital motion, auroras, shooting stars, galaxies, and space debris. These features highlight OpenGL's ability to simulate imaginative phenomena.
10. Enhanced Mode – Adds additional graphical polish such as glowing star effects, more complex gradients, and additional transparency layers.

The combination of these features ensures that the simulation is not only educational but also visually engaging. Each feature was carefully integrated to demonstrate different aspects of OpenGL rendering.

The "Finding Island" simulation offers a wide range of features that demonstrate the versatility of OpenGL.

1. Day and Night Cycle – The system transitions between day and night using different sky gradients, replacing the sun with the moon, and enabling/disabling stars and birds accordingly.
2. Animated Clouds and Birds – Clouds drift across the sky at different speeds, while birds flap their wings and move along parabolic paths, active only during the day.
3. Water and Waves – The water body is animated with sine waves that simulate realistic wave motion. Reflections of the moon are also added at night.
4. Boats – A fishing boat and cargo boat move across the water, with bobbing effects to simulate buoyancy. Human figures are also rendered on the boats to enhance realism.
5. Cars and Roads – Cars of different colors and sizes move along the road, while an airplane flies across the sky, adding dynamism to the man-made scene.
6. Village and House – A small village environment with houses and trees is constructed to ground the scene in a natural rural setting.
7. Bridge Scene – A separate scenario features a bridge over water, complete with lighting effects, structural supports, and traffic.
8. Island Scene – The island features palm trees, landmasses, and greenery during normal mode, while in space mode it transforms into a futuristic scene with crystals and portals.
9. Space Mode – Includes planets with orbital motion, auroras, shooting stars, galaxies, and space debris. These features highlight OpenGL's ability to simulate imaginative phenomena.
10. Enhanced Mode – Adds additional graphical polish such as glowing star effects, more complex gradients, and additional transparency layers.

The combination of these features ensures that the simulation is not only educational but also visually engaging. Each feature was carefully integrated to demonstrate different

aspects of OpenGL rendering.

The “Finding Island” simulation offers a wide range of features that demonstrate the versatility of OpenGL.

1. Day and Night Cycle – The system transitions between day and night using different sky gradients, replacing the sun with the moon, and enabling/disabling stars and birds accordingly.
2. Animated Clouds and Birds – Clouds drift across the sky at different speeds, while birds flap their wings and move along parabolic paths, active only during the day.
3. Water and Waves – The water body is animated with sine waves that simulate realistic wave motion. Reflections of the moon are also added at night.
4. Boats – A fishing boat and cargo boat move across the water, with bobbing effects to simulate buoyancy. Human figures are also rendered on the boats to enhance realism.
5. Cars and Roads – Cars of different colors and sizes move along the road, while an airplane flies across the sky, adding dynamism to the man-made scene.
6. Village and House – A small village environment with houses and trees is constructed to ground the scene in a natural rural setting.
7. Bridge Scene – A separate scenario features a bridge over water, complete with lighting effects, structural supports, and traffic.
8. Island Scene – The island features palm trees, landmasses, and greenery during normal mode, while in space mode it transforms into a futuristic scene with crystals and portals.
9. Space Mode – Includes planets with orbital motion, auroras, shooting stars, galaxies, and space debris. These features highlight OpenGL’s ability to simulate imaginative phenomena.
10. Enhanced Mode – Adds additional graphical polish such as glowing star effects, more complex gradients, and additional transparency layers.

The combination of these features ensures that the simulation is not only educational but also visually engaging. Each feature was carefully integrated to demonstrate different aspects of OpenGL rendering.

Future Directions

While “Finding Island” already demonstrates an impressive set of features, there is significant room for future development.

One of the primary directions is user interactivity. Currently, the simulation is automatic, with objects moving independently of user control. Adding keyboard and mouse

interactions could allow users to control boats, cars, or even launch new shooting stars. This would transform the project from a passive simulation into an interactive experience.

Another future direction is the integration of modern OpenGL features such as shaders and textures. By using GLSL shaders, lighting effects could be made more realistic, water could include reflections and refractions, and planets could display surface textures. This would greatly increase the realism of the simulation.

The project could also be extended into 3D space using perspective projections instead of orthographic projections. This would enable features such as rotating cameras, zoom effects, and more immersive environments. Eventually, such a system could be ported into VR or AR platforms, giving users the ability to “walk” through the village, fly over the bridge, or explore the space environment.

Additionally, sound effects could be integrated to accompany animations. Birds chirping, cars honking, waves crashing, or space ambience could make the experience more engaging.

In conclusion, future directions include adding interactivity, modern OpenGL features, 3D rendering, VR/AR support, and sound integration.

While “Finding Island” already demonstrates an impressive set of features, there is significant room for future development.

One of the primary directions is user interactivity. Currently, the simulation is automatic, with objects moving independently of user control. Adding keyboard and mouse interactions could allow users to control boats, cars, or even launch new shooting stars. This would transform the project from a passive simulation into an interactive experience.

Another future direction is the integration of modern OpenGL features such as shaders and textures. By using GLSL shaders, lighting effects could be made more realistic, water could include reflections and refractions, and planets could display surface textures. This would greatly increase the realism of the simulation.

The project could also be extended into 3D space using perspective projections instead of orthographic projections. This would enable features such as rotating cameras, zoom effects, and more immersive environments. Eventually, such a system could be ported into VR or AR platforms, giving users the ability to “walk” through the village, fly over

the bridge, or explore the space environment.

Additionally, sound effects could be integrated to accompany animations. Birds chirping, cars honking, waves crashing, or space ambience could make the experience more engaging.

In conclusion, future directions include adding interactivity, modern OpenGL features, 3D rendering, VR/AR support, and sound integration.

While “Finding Island” already demonstrates an impressive set of features, there is significant room for future development.

One of the primary directions is user interactivity. Currently, the simulation is automatic, with objects moving independently of user control. Adding keyboard and mouse interactions could allow users to control boats, cars, or even launch new shooting stars. This would transform the project from a passive simulation into an interactive experience.

Another future direction is the integration of modern OpenGL features such as shaders and textures. By using GLSL shaders, lighting effects could be made more realistic, water could include reflections and refractions, and planets could display surface textures. This would greatly increase the realism of the simulation.

The project could also be extended into 3D space using perspective projections instead of orthographic projections. This would enable features such as rotating cameras, zoom effects, and more immersive environments. Eventually, such a system could be ported into VR or AR platforms, giving users the ability to “walk” through the village, fly over the bridge, or explore the space environment.

Additionally, sound effects could be integrated to accompany animations. Birds chirping, cars honking, waves crashing, or space ambience could make the experience more engaging.

In conclusion, future directions include adding interactivity, modern OpenGL features, 3D rendering, VR/AR support, and sound integration.

While “Finding Island” already demonstrates an impressive set of features, there is significant room for future development.

One of the primary directions is user interactivity. Currently, the simulation is automatic,

with objects moving independently of user control. Adding keyboard and mouse interactions could allow users to control boats, cars, or even launch new shooting stars. This would transform the project from a passive simulation into an interactive experience.

Another future direction is the integration of modern OpenGL features such as shaders and textures. By using GLSL shaders, lighting effects could be made more realistic, water could include reflections and refractions, and planets could display surface textures. This would greatly increase the realism of the simulation.

The project could also be extended into 3D space using perspective projections instead of orthographic projections. This would enable features such as rotating cameras, zoom effects, and more immersive environments. Eventually, such a system could be ported into VR or AR platforms, giving users the ability to “walk” through the village, fly over the bridge, or explore the space environment.

Additionally, sound effects could be integrated to accompany animations. Birds chirping, cars honking, waves crashing, or space ambience could make the experience more engaging.

In conclusion, future directions include adding interactivity, modern OpenGL features, 3D rendering, VR/AR support, and sound integration.

While “Finding Island” already demonstrates an impressive set of features, there is significant room for future development.

One of the primary directions is user interactivity. Currently, the simulation is automatic, with objects moving independently of user control. Adding keyboard and mouse interactions could allow users to control boats, cars, or even launch new shooting stars. This would transform the project from a passive simulation into an interactive experience.

Another future direction is the integration of modern OpenGL features such as shaders and textures. By using GLSL shaders, lighting effects could be made more realistic, water could include reflections and refractions, and planets could display surface textures. This would greatly increase the realism of the simulation.

The project could also be extended into 3D space using perspective projections instead of orthographic projections. This would enable features such as rotating cameras, zoom effects, and more immersive environments. Eventually, such a system could be ported

into VR or AR platforms, giving users the ability to “walk” through the village, fly over the bridge, or explore the space environment.

Additionally, sound effects could be integrated to accompany animations. Birds chirping, cars honking, waves crashing, or space ambience could make the experience more engaging.

In conclusion, future directions include adding interactivity, modern OpenGL features, 3D rendering, VR/AR support, and sound integration.

Conclusion

The “Finding Island” project demonstrates the ability of OpenGL to create visually engaging and educational simulations. By integrating multiple scenarios, day and night cycles, natural and man-made elements, and even futuristic space environments, the project highlights the versatility of graphics programming.

One of the most significant achievements of the project is the modular design, which ensures clarity, scalability, and extensibility. Each function and struct is designed to handle a specific element, making it easy to maintain and expand. The synchronization of animations also shows the importance of timing and coordination in graphical systems.

The project is valuable not only as a learning tool for computer graphics students but also as a foundation for interactive entertainment applications. It bridges the gap between academic exercises and creative storytelling.

In conclusion, “Finding Island” is more than a graphics assignment—it is a comprehensive simulation system that demonstrates technical proficiency, artistic creativity, and practical application of computer graphics concepts.

The “Finding Island” project demonstrates the ability of OpenGL to create visually engaging and educational simulations. By integrating multiple scenarios, day and night cycles, natural and man-made elements, and even futuristic space environments, the project highlights the versatility of graphics programming.

One of the most significant achievements of the project is the modular design, which ensures clarity, scalability, and extensibility. Each function and struct is designed to handle a specific element, making it easy to maintain and expand. The synchronization of animations also shows the importance of timing and coordination in graphical systems.

The project is valuable not only as a learning tool for computer graphics students but also as a foundation for interactive entertainment applications. It bridges the gap between academic exercises and creative storytelling.

In conclusion, “Finding Island” is more than a graphics assignment—it is a comprehensive simulation system that demonstrates technical proficiency, artistic creativity, and practical application of computer graphics concepts.

The “Finding Island” project demonstrates the ability of OpenGL to create visually engaging and educational simulations. By integrating multiple scenarios, day and night cycles, natural and man-made elements, and even futuristic space environments, the project highlights the versatility of graphics programming.

One of the most significant achievements of the project is the modular design, which ensures clarity, scalability, and extensibility. Each function and struct is designed to handle a specific element, making it easy to maintain and expand. The synchronization of animations also shows the importance of timing and coordination in graphical systems.

The project is valuable not only as a learning tool for computer graphics students but also as a foundation for interactive entertainment applications. It bridges the gap between academic exercises and creative storytelling.

In conclusion, “Finding Island” is more than a graphics assignment—it is a comprehensive simulation system that demonstrates technical proficiency, artistic creativity, and practical application of computer graphics concepts.

The “Finding Island” project demonstrates the ability of OpenGL to create visually engaging and educational simulations. By integrating multiple scenarios, day and night cycles, natural and man-made elements, and even futuristic space environments, the project highlights the versatility of graphics programming.

One of the most significant achievements of the project is the modular design, which ensures clarity, scalability, and extensibility. Each function and struct is designed to handle a specific element, making it easy to maintain and expand. The synchronization of animations also shows the importance of timing and coordination in graphical systems.

The project is valuable not only as a learning tool for computer graphics students but also as a foundation for interactive entertainment applications. It bridges the gap between academic exercises and creative storytelling.

In conclusion, “Finding Island” is more than a graphics assignment—it is a comprehensive simulation system that demonstrates technical proficiency, artistic creativity, and practical application of computer graphics concepts.

The “Finding Island” project demonstrates the ability of OpenGL to create visually engaging and educational simulations. By integrating multiple scenarios, day and night cycles, natural and man-made elements, and even futuristic space environments, the project highlights the versatility of graphics programming.

One of the most significant achievements of the project is the modular design, which ensures clarity, scalability, and extensibility. Each function and struct is designed to handle a specific element, making it easy to maintain and expand. The synchronization of animations also shows the importance of timing and coordination in graphical systems.

The project is valuable not only as a learning tool for computer graphics students but also as a foundation for interactive entertainment applications. It bridges the gap between academic exercises and creative storytelling.

In conclusion, “Finding Island” is more than a graphics assignment—it is a comprehensive simulation system that demonstrates technical proficiency, artistic creativity, and practical application of computer graphics concepts.

Implementation code:

```
#include <windows.h>

#include <GL/glut.h>

#include <cmath>

#include <vector>

#include <cstdlib>

#include <ctime>

#include <iostream>


const int WINDOW_WIDTH = 1000;

const int WINDOW_HEIGHT = 700;
```

```
float sunX = 870;
float sunY = 560;
float sunSpeed = 0.22f;
float sunAngle = 30.0f * 3.14159f / 180.0f;

float cloud1X = 160, cloud2X = 300, cloud3X = 520, cloud4X = 720;

float bird1X = 200, bird1Y = 620;
float bird2X = 400, bird2Y = 650;
float bird3X = 600, bird3Y = 630;

float waveTime = 0.0f;

float redCarX = -250.0f, greenCarX = -600.0f, blueCarX = -1000.0f;
float redCarV = 2.4f, greenCarV = 3.1f, blueCarV = 2.8f;

float fishingBoatX = -300.0f, cargoBoatX = -900.0f;
float fishingBoatSpeed = 1.2f;
float cargoBoatSpeed = 1.5f;
float fishingBoatV = fishingBoatSpeed;
float cargoBoatV = cargoBoatSpeed;

float airplaneX = -200;
float airplaneY = 650;
```

```
bool isDay = true;
bool enhancedMode = false;
bool spaceMode = false;
float globalTime = 0.0f;

int currentScenario = 0;

std::vector<std::pair<float, float> > stars;
std::vector<std::pair<float, float> > twinklePhases;

struct ShootingStar {
    float startX, startY;
    float endX, endY;
    float currentX, currentY;
    float speed;
    float trailLength;
    float life;
    float maxLife;
    bool active;
};

struct Planet {
    float x, y;
    float radius;
```

```
    float orbitRadius;
    float orbitSpeed;
    float angle;
    float r, g, b;
    float rings;
    bool active;
};

struct Aurora {
    float x, y;
    float width, height;
    float waveOffset;
    float intensity;
    float r, g, b;
};

struct SpaceDebris {
    float x, y;
    float vx, vy;
    float rotation;
    float size;
    float life;
    bool active;
};
```

```
std::vector<ShootingStar> shootingStars;
std::vector<Planet> planets;
std::vector<Aurora> auroras;
std::vector<SpaceDebris> spaceDebris;

int transitionLock = 0;
int shootingStarTimer = 0;

// Function declarations
void drawShootingStars();
void updateShootingStars();
void createShootingStar();

void initStars(int count = 100) {
    stars.clear();
    twinklePhases.clear();
    for (int i = 0; i < count; ++i) {
        float x = rand() % WINDOW_WIDTH;
        float y = 260 + (rand() % (WINDOW_HEIGHT - 260));
        stars.push_back(std::make_pair(x, y));

        float phase = (rand() % 628) / 100.0f;
        float speed = 0.5f + (rand() % 100) / 100.0f;
        twinklePhases.push_back(std::make_pair(phase, speed));
    }
```

```
}

void initSpaceElements() {
    planets.clear();
    auroras.clear();
    spaceDebris.clear();

    // Planet 1 - Mars-like
    Planet p1;
    p1.x = WINDOW_WIDTH * 0.15f;
    p1.y = WINDOW_HEIGHT * 0.8f;
    p1.radius = 60;
    p1.orbitRadius = 30;
    p1.orbitSpeed = 0.005f;
    p1.angle = 0;
    p1.r = 0.9f; p1.g = 0.4f; p1.b = 0.2f;
    p1.rings = 0;
    p1.active = true;
    planets.push_back(p1);

    // Planet 2 - Saturn-like
    Planet p2;
    p2.x = WINDOW_WIDTH * 0.8f;
    p2.y = WINDOW_HEIGHT * 0.7f;
    p2.radius = 45;
```

```
p2.orbitRadius = 20;
p2.orbitSpeed = 0.008f;
p2.angle = 3.14f;
p2.r = 0.2f; p2.g = 0.6f; p2.b = 0.9f;
p2.rings = 1;
p2.active = true;
planets.push_back(p2);

// Planet 3 - Purple planet
Planet p3;
p3.x = WINDOW_WIDTH * 0.6f;
p3.y = WINDOW_HEIGHT * 0.9f;
p3.radius = 25;
p3.orbitRadius = 15;
p3.orbitSpeed = 0.012f;
p3.angle = 1.57f;
p3.r = 0.7f; p3.g = 0.3f; p3.b = 0.8f;
p3.rings = 0;
p3.active = true;
planets.push_back(p3);

// Initialize auroras
for (int i = 0; i < 3; ++i) {
    Aurora a;
    a.x = 100 + i * 300;
```

```
        a.y = WINDOW_HEIGHT - 100;
        a.width = 150;
        a.height = 80;
        a.waveOffset = i * 2.0f;
        a.intensity = 0.6f;
        a.r = 0.2f + i * 0.3f;
        a.g = 0.8f - i * 0.2f;
        a.b = 0.4f + i * 0.4f;
        auroras.push_back(a);
    }

    // Initialize space debris
    for (int i = 0; i < 8; ++i) {
        SpaceDebris d;
        d.x = rand() % WINDOW_WIDTH;
        d.y = WINDOW_HEIGHT + rand() % 200;
        d.vx = (rand() % 100 - 50) / 100.0f;
        d.vy = -(rand() % 100 + 50) / 100.0f;
        d.rotation = 0;
        d.size = 5 + rand() % 15;
        d.life = 300 + rand() % 200;
        d.active = true;
        spaceDebris.push_back(d);
    }
}
```

```
void init2D() {
    glClearColor(0.52f, 0.80f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, WINDOW_WIDTH, 0, WINDOW_HEIGHT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
}

inline void setColor(float r, float g, float b, float a = 1.0f) {
    glColor4f(r, g, b, a);
}

void createShootingStar() {
    if (shootingStars.size() >= 3) return;

    ShootingStar star;

    if (rand() % 2 == 0) {
        star.startX = -50 + (rand() % 200);
        star.startY = WINDOW_HEIGHT - 50 + (rand() % 100);
```

```
        star.endX = WINDOW_WIDTH + 100;
        star.endY = 300 + (rand() % 200);
    } else {
        star.startX = WINDOW_WIDTH + 50 - (rand() % 200);
        star.startY = WINDOW_HEIGHT - 50 + (rand() % 100);
        star.endX = -100;
        star.endY = 300 + (rand() % 200);
    }

    star.currentX = star.startX;
    star.currentY = star.startY;
    star.speed = 3.0f + (rand() % 5);
    star.trailLength = 80.0f + (rand() % 40);
    star.life = 0.0f;
    star.maxLife = 180.0f + (rand() % 120);
    star.active = true;

    shootingStars.push_back(star);
}

void updateShootingStars() {
    if (!isDay) {
        shootingStarTimer++;
        if (shootingStarTimer > 300 + (rand() % 600)) {
            createShootingStar();
```

```
        shootingStarTimer = 0;
    }
}

for (size_t i = 0; i < shootingStars.size(); ++i) {
    if (!shootingStars[i].active) continue;

    ShootingStar &star = shootingStars[i];
    star.life += 1.0f;

    if (star.life > star.maxLife) {
        star.active = false;
        continue;
    }

    float dx = star.endX - star.startX;
    float dy = star.endY - star.startY;

    float progress = star.life / star.maxLife;
    star.currentX = star.startX + (dx * progress);
    star.currentY = star.startY + (dy * progress);

    if (star.currentX < -200 || star.currentX > WINDOW_WIDTH + 200 ||
        star.currentY < -100 || star.currentY > WINDOW_HEIGHT + 100) {
        star.active = false;
```

```
        }
    }

    for (size_t i = shootingStars.size(); i > 0; --i) {
        if (!shootingStars[i-1].active) {
            shootingStars.erase(shootingStars.begin() + (i-1));
        }
    }
}

void updateSpaceElements() {
    if (!spaceMode) return;

    for (size_t i = 0; i < planets.size(); ++i) {
        planets[i].angle += planets[i].orbitSpeed;
        if (planets[i].angle > 6.28f) planets[i].angle -= 6.28f;
    }

    for (size_t i = 0; i < auroras.size(); ++i) {
        auroras[i].waveOffset += 0.02f;
        auroras[i].intensity = 0.4f + 0.3f * sinf(globalTime * 0.01f + i);
    }

    for (size_t i = 0; i < spaceDebris.size(); ++i) {
        if (!spaceDebris[i].active) continue;
```

```
        spaceDebris[i].x += spaceDebris[i].vx;
        spaceDebris[i].y += spaceDebris[i].vy;
        spaceDebris[i].rotation += 0.05f;
        spaceDebris[i].life -= 1.0f;

        if (spaceDebris[i].life <= 0 || spaceDebris[i].y < -50) {
            spaceDebris[i].x = rand() % WINDOW_WIDTH;
            spaceDebris[i].y = WINDOW_HEIGHT + rand() % 200;
            spaceDebris[i].vx = (rand() % 100 - 50) / 100.0f;
            spaceDebris[i].vy = -(rand() % 100 + 50) / 100.0f;
            spaceDebris[i].life = 300 + rand() % 200;
        }
    }
}

void drawShootingStars() {
    if (isDay) return;

    for (size_t i = 0; i < shootingStars.size(); ++i) {
        const ShootingStar &star = shootingStars[i];
        if (!star.active) continue;

        float alpha = 1.0f - (star.life / star.maxLife);
        if (alpha < 0.0f) alpha = 0.0f;
```

```
        if (alpha > 1.0f) alpha = 1.0f;

        float dx = star.endX - star.startX;
        float dy = star.endY - star.startY;
        float distance = sqrtf(dx * dx + dy * dy);

        if (distance > 0) {
            dx /= distance;
            dy /= distance;

            int trailSegments = enhancedMode ? 30 : 20;
            for (int j = 0; j < trailSegments; ++j) {
                float segmentAlpha = alpha * (1.0f - (float)j / trailSegments);
                float segmentDistance = (star.trailLength * j) / trailSegments;

                float trailX = star.currentX - (dx * segmentDistance);
                float trailY = star.currentY - (dy * segmentDistance);

                setColor(1.0f * segmentAlpha, 1.0f * segmentAlpha, 0.8f * segmentAlpha,
segmentAlpha);

                float size = enhancedMode ? 4.0f * (1.0f - (float)j / trailSegments)
                                          : 3.0f * (1.0f - (float)j / trailSegments);

                glBegin(GL_QUADS);
```

```
            glVertex2f(trailX - size, trailY - size);
            glVertex2f(trailX + size, trailY - size);
            glVertex2f(trailX + size, trailY + size);
            glVertex2f(trailX - size, trailY + size);
            glEnd();

            if (enhancedMode && j < trailSegments / 2) {
                setColor(0.8f * segmentAlpha, 0.9f * segmentAlpha, 1.0f * segmentAlpha,
segmentAlpha * 0.5f);
                glBegin(GL_QUADS);
                float glowSize = size * 1.8f;
                glVertex2f(trailX - glowSize, trailY - glowSize);
                glVertex2f(trailX + glowSize, trailY - glowSize);
                glVertex2f(trailX + glowSize, trailY + glowSize);
                glVertex2f(trailX - glowSize, trailY + glowSize);
                glEnd();
            }
        }

        if (enhancedMode) {
            setColor(1.0f * alpha, 1.0f * alpha, 0.7f * alpha, alpha * 0.3f);
            glBegin(GL_TRIANGLE_FAN);
            glVertex2f(star.currentX, star.currentY);
            for (int s = 0; s <= 16; ++s) {
                float ang = 2.0f * 3.1415926f * s / 16;
```

```
                glVertex2f(star.currentX + 12.0f * cosf(ang), star.currentY + 12.0f *
sinf(ang));
            }
            glEnd();
        }

        setColor(1.0f * alpha, 1.0f * alpha, 0.9f * alpha, alpha);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(star.currentX, star.currentY);
        for (int s = 0; s <= 16; ++s) {
            float ang = 2.0f * 3.1415926f * s / 16;
            glVertex2f(star.currentX + 6.0f * cosf(ang), star.currentY + 6.0f * sinf(ang));
        }
        glEnd();
    }
  }
}

void drawNebulaBackground() {
  if (!spaceMode) return;

  for (int layer = 0; layer < 3; ++layer) {
    setColor(0.3f + layer * 0.1f, 0.1f + layer * 0.2f, 0.6f + layer * 0.1f, 0.1f);

    for (int x = 0; x < WINDOW_WIDTH; x += 50) {
```

```
            for (int y = 260; y < WINDOW_HEIGHT; y += 50) {
                float size = 40 + 20 * sinf(x * 0.01f + y * 0.008f + globalTime * 0.005f +
layer);

                glBegin(GL_TRIANGLE_FAN);
                glVertex2f(x, y);
                for (int s = 0; s <= 20; ++s) {
                    float ang = 2.0f * 3.1415926f * s / 20;
                    glVertex2f(x + size * cosf(ang), y + size * sinf(ang));
                }
                glEnd();
            }
        }
    }
}

void drawDistantGalaxy() {
    if (!spaceMode) return;

    float galaxyX = WINDOW_WIDTH * 0.85f;
    float galaxyY = WINDOW_HEIGHT * 0.85f;
    float galaxyRadius = 80;

    setColor(0.8f, 0.6f, 1.0f, 0.4f);
```

```
glBegin(GL_TRIANGLE_FAN);
glVertex2f(galaxyX, galaxyY);
for (int s = 0; s <= 30; ++s) {
    float ang = 2.0f * 3.1415926f * s / 30;
    glVertex2f(galaxyX + 20 * cosf(ang), galaxyY + 20 * sinf(ang));
}
glEnd();

setColor(0.6f, 0.4f, 0.8f, 0.2f);
for (int arm = 0; arm < 3; ++arm) {
    glBegin(GL_QUAD_STRIP);
    for (int i = 0; i <= 60; ++i) {
        float t = i / 60.0f;
        float angle = arm * 2.09f + t * 6.28f + globalTime * 0.002f;
        float radius = t * galaxyRadius;

        float x1 = galaxyX + radius * cosf(angle);
        float y1 = galaxyY + radius * sinf(angle);
        float x2 = galaxyX + (radius + 5) * cosf(angle);
        float y2 = galaxyY + (radius + 5) * sinf(angle);

        glVertex2f(x1, y1);
        glVertex2f(x2, y2);
    }
    glEnd();
```

```
    }
}

void drawPlanets() {
    if (!spaceMode) return;

    for (size_t i = 0; i < planets.size(); ++i) {
        const Planet &p = planets[i];
        if (!p.active) continue;

        float orbitalX = p.x + p.orbitRadius * cosf(p.angle);
        float orbitalY = p.y + p.orbitRadius * sinf(p.angle);

        if (enhancedMode) {
            setColor(p.r, p.g, p.b, 0.3f);
            glBegin(GL_TRIANGLE_FAN);
            glVertex2f(orbitalX, orbitalY);
            for (int s = 0; s <= 60; ++s) {
                float ang = 2.0f * 3.1415926f * s / 60;
                glVertex2f(orbitalX + (p.radius * 1.5f) * cosf(ang),
                        orbitalY + (p.radius * 1.5f) * sinf(ang));
            }
            glEnd();
        }
```

```
setColor(p.r, p.g, p.b);
glBegin(GL_TRIANGLE_FAN);
glVertex2f(orbitalX, orbitalY);
for (int s = 0; s <= 60; ++s) {
   float ang = 2.0f * 3.1415926f * s / 60;
   glVertex2f(orbitalX + p.radius * cosf(ang),
         orbitalY + p.radius * sinf(ang));
}
glEnd();

setColor(p.r * 0.7f, p.g * 0.7f, p.b * 0.7f);
for (int j = 0; j < 3; ++j) {
   float craterX = orbitalX + (p.radius * 0.3f) * cosf(j * 2.0f + p.angle);
   float craterY = orbitalY + (p.radius * 0.3f) * sinf(j * 2.0f + p.angle);
   float craterSize = p.radius * 0.15f;

   glBegin(GL_TRIANGLE_FAN);
   glVertex2f(craterX, craterY);
   for (int s = 0; s <= 20; ++s) {
      float ang = 2.0f * 3.1415926f * s / 20;
      glVertex2f(craterX + craterSize * cosf(ang),
            craterY + craterSize * sinf(ang));
   }
   glEnd();
}
```

```
        if (p.rings > 0) {
            setColor(p.r * 0.8f, p.g * 0.8f, p.b * 0.8f, 0.7f);

            for (int ring = 0; ring < 3; ++ring) {
                float ringRadius = p.radius * (1.3f + ring * 0.2f);
                float ringThickness = 3.0f;

                glBegin(GL_QUAD_STRIP);
                for (int s = 0; s <= 60; ++s) {
                    float ang = 2.0f * 3.1415926f * s / 60;
                    float innerR = ringRadius - ringThickness;
                    float outerR = ringRadius + ringThickness;

                    glVertex2f(orbitalX + innerR * cosf(ang),
                            orbitalY + innerR * sinf(ang));
                    glVertex2f(orbitalX + outerR * cosf(ang),
                            orbitalY + outerR * sinf(ang));
                }
                glEnd();
            }
        }
    }
}
```

```
void drawAuroras() {
    if (!spaceMode) return;

    for (size_t i = 0; i < auroras.size(); ++i) {
        const Aurora &a = auroras[i];

        setColor(a.r, a.g, a.b, a.intensity * 0.6f);

        glBegin(GL_QUAD_STRIP);
        for (int x = 0; x <= a.width; x += 10) {
            float wave1 = a.height * 0.3f * sinf(x * 0.02f + a.waveOffset);
            float wave2 = a.height * 0.5f * sinf(x * 0.03f + a.waveOffset * 1.5f);

            float y1 = a.y + wave1;
            float y2 = a.y + a.height + wave2;

            float fadeAlpha = a.intensity * (1.0f - (float)x / a.width);
            setColor(a.r, a.g, a.b, fadeAlpha * 0.4f);

            glVertex2f(a.x + x, y1);
            glVertex2f(a.x + x, y2);
        }
        glEnd();
    }
}
```

```
void drawSpaceDebris() {
    if (!spaceMode) return;

    for (size_t i = 0; i < spaceDebris.size(); ++i) {
        const SpaceDebris &d = spaceDebris[i];
        if (!d.active) continue;

        glPushMatrix();
        glTranslatef(d.x, d.y, 0);
        glRotatef(d.rotation * 180.0f / 3.14159f, 0, 0, 1);

        setColor(0.6f, 0.5f, 0.4f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(0, 0);
        for (int s = 0; s <= 8; ++s) {
            float ang = 2.0f * 3.1415926f * s / 8;
            float radius = d.size * (0.7f + 0.3f * sinf(s * 1.5f));
            glVertex2f(radius * cosf(ang), radius * sinf(ang));
        }
        glEnd();

        setColor(0.4f, 0.3f, 0.2f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(d.size * 0.3f, d.size * 0.2f);
```

```
        for (int s = 0; s <= 6; ++s) {
            float ang = 2.0f * 3.1415926f * s / 6;
            float radius = d.size * 0.2f;
            glVertex2f(d.size * 0.3f + radius * cosf(ang),
                    d.size * 0.2f + radius * sinf(ang));
        }
        glEnd();

        glPopMatrix();
    }
}

void drawSky() {
    if (enhancedMode) {
        glBegin(GL_QUADS);

        if (isDay) {
            setColor(0.2f, 0.6f, 1.0f);
            glVertex2f(0, WINDOW_HEIGHT);
            glVertex2f(WINDOW_WIDTH, WINDOW_HEIGHT);

            setColor(0.7f, 0.85f, 1.0f);
            glVertex2f(WINDOW_WIDTH, 260);
            glVertex2f(0, 260);
        } else {
```

```
        float auroraEffect = sinf(globalTime * 0.01f) * 0.1f;
        setColor(0.02f + auroraEffect * 0.03f, 0.02f + auroraEffect * 0.05f, 0.08f + auroraEffect * 0.07f);
        glVertex2f(0, WINDOW_HEIGHT);
        glVertex2f(WINDOW_WIDTH, WINDOW_HEIGHT);

        setColor(0.05f + auroraEffect * 0.05f, 0.05f + auroraEffect * 0.03f, 0.15f + auroraEffect * 0.05f);
        glVertex2f(WINDOW_WIDTH, 260);
        glVertex2f(0, 260);
    }
    glEnd();

    if (!isDay) {
        for (size_t i = 0; i < stars.size(); ++i) {
            float x = stars[i].first, y = stars[i].second;
            float phase = twinklePhases[i].first;
            float speed = twinklePhases[i].second;

            float brightness = 0.7f + 0.3f * sinf(globalTime * 0.02f * speed + phase);
            float size = 1.5f + brightness * 1.5f;

            setColor(1.0f, 1.0f, 1.0f, brightness);

            glBegin(GL_QUADS);
            glVertex2f(x - size, y - size);
```

```
            glVertex2f(x + size, y - size);
            glVertex2f(x + size, y + size);
            glVertex2f(x - size, y + size);
            glEnd();

            if (brightness > 0.9f) {
                setColor(0.8f, 0.9f, 1.0f, (brightness - 0.9f) * 3.0f);
                float sparkleSize = size * 2.0f;
                glBegin(GL_QUADS);
                glVertex2f(x - sparkleSize, y - sparkleSize);
                glVertex2f(x + sparkleSize, y - sparkleSize);
                glVertex2f(x + sparkleSize, y + sparkleSize);
                glVertex2f(x - sparkleSize, y + sparkleSize);
                glEnd();
            }
        }
    }
} else {
    if (isDay)
        setColor(0.55f, 0.8f, 1.0f);
    else
        setColor(0.05f, 0.05f, 0.12f);

    glBegin(GL_QUADS);
    glVertex2f(0, 260);
```

```
        glVertex2f(WINDOW_WIDTH, 260);
        glVertex2f(WINDOW_WIDTH, WINDOW_HEIGHT);
        glVertex2f(0, WINDOW_HEIGHT);
        glEnd();

        if (!isDay) {
            setColor(1, 1, 1);
            int segs = 10;
            for (size_t i = 0; i < stars.size(); ++i) {
                float x = stars[i].first, y = stars[i].second;
                glBegin(GL_TRIANGLE_FAN);
                glVertex2f(x, y);
                for (int s = 0; s <= segs; ++s) {
                    float ang = 2.0f * 3.1415926f * s / segs;
                    glVertex2f(x + 2.0f * cosf(ang), y + 2.0f * sinf(ang));
                }
                glEnd();
            }
        }
    }

    if (!isDay) {
        drawShootingStars();
    }
}
```

```
void drawSunOrMoon() {
    int segs = 60;
    float r = 45;

    if (isDay) {
        if (enhancedMode) {
            setColor(1.0f, 0.8f, 0.3f, 0.3f);
            for (int i = 0; i < 12; ++i) {
                float angle = i * 30.0f * 3.14159f / 180.0f + globalTime * 0.01f;
                float rayLength = 80.0f + sinf(globalTime * 0.02f + i) * 20.0f;

                glBegin(GL_TRIANGLES);
                glVertex2f(sunX, sunY);
                glVertex2f(sunX + rayLength * cosf(angle), sunY + rayLength * sinf(angle));
                glVertex2f(sunX + rayLength * cosf(angle + 0.1f), sunY + rayLength *
sinf(angle + 0.1f));
                glEnd();
            }

            setColor(1.0f, 0.7f, 0.2f, 0.4f);
            glBegin(GL_TRIANGLE_FAN);
            glVertex2f(sunX, sunY);
            for (int s = 0; s <= segs; ++s) {
                float ang = 2.0f * 3.1415926f * s / segs;
```

```
            glVertex2f(sunX + (r * 1.6f) * cosf(ang), sunY + (r * 1.6f) * sinf(ang));
        }
        glEnd();
    }

    setColor(1.0f, 0.9f, 0.3f);
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(sunX, sunY);
    for (int s = 0; s <= segs; ++s) {
        float ang = 2.0f * 3.1415926f * s / segs;
        glVertex2f(sunX + r * cosf(ang), sunY + r * sinf(ang));
    }
    glEnd();
} else {
    if (enhancedMode) {
        setColor(0.7f, 0.7f, 0.9f, 0.3f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(sunX, sunY);
        for (int s = 0; s <= segs; ++s) {
            float ang = 2.0f * 3.1415926f * s / segs;
            glVertex2f(sunX + (r * 1.8f) * cosf(ang), sunY + (r * 1.8f) * sinf(ang));
        }
        glEnd();
    }
```

```
        setColor(0.9f, 0.9f, 1.0f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(sunX, sunY);
        for (int s = 0; s <= segs; ++s) {
            float ang = 2.0f * 3.1415926f * s / segs;
            glVertex2f(sunX + r * cosf(ang), sunY + r * sinf(ang));
        }
        glEnd();

        setColor(0.05f, 0.05f, 0.12f);
        float cx = sunX + r * 0.4f;
        float cy = sunY + r * 0.1f;
        float rr = r * 0.7f;
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(cx, cy);
        for (int s = 0; s <= segs; ++s) {
            float ang = 2.0f * 3.1415926f * s / segs;
            glVertex2f(cx + rr * cosf(ang), cy + rr * sinf(ang));
        }
        glEnd();
    }
}

void drawClouds() {
    struct Part {
```

```
    float ox, oy, rad;
};
Part cloudParts[4] = {{0, 0, 25}, {25, 10, 22}, {-25, 10, 22}, {48, 0, 18}};

float cloudPos[4][3] = {
    {cloud1X, 620, 1.1f},
    {cloud2X, 650, 0.9f},
    {cloud3X, 620, 1.0f},
    {cloud4X, 640, 1.2f}
};

for (int c = 0; c < 4; ++c) {
    if (enhancedMode) {
        for (int i = 0; i < 4; ++i) {
            float cx = cloudPos[c][0] + cloudParts[i].ox * cloudPos[c][2] + 5;
            float cy = cloudPos[c][1] + cloudParts[i].oy * cloudPos[c][2] - 5;
            float radius = cloudParts[i].rad * cloudPos[c][2];

            setColor(0.6f, 0.6f, 0.7f, 0.3f);
            glBegin(GL_TRIANGLE_FAN);
            glVertex2f(cx, cy);
            for (int s = 0; s <= 40; ++s) {
                float ang = 2.0f * 3.1415926f * s / 40;
                glVertex2f(cx + radius * cosf(ang), cy + radius * sinf(ang));
            }
```

```
            glEnd();
        }
    }

    for (int i = 0; i < 4; ++i) {
        float cx = cloudPos[c][0] + cloudParts[i].ox * cloudPos[c][2];
        float cy = cloudPos[c][1] + cloudParts[i].oy * cloudPos[c][2];
        float radius = cloudParts[i].rad * cloudPos[c][2];

        if (!isDay)
            setColor(0.6f, 0.6f, 0.65f);
        else
            setColor(1, 1, 1);

        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(cx, cy);
        for (int s = 0; s <= 40; ++s) {
            float ang = 2.0f * 3.1415926f * s / 40;
            glVertex2f(cx + radius * cosf(ang), cy + radius * sinf(ang));
        }
        glEnd();
    }
  }
}
```

```
void drawBirds() {
    if (!isDay)
        return;

    setColor(0, 0, 0);

    if (enhancedMode) {
        float wingFlap = sinf(globalTime * 0.1f) * 0.3f;

        glBegin(GL_LINES);
        glVertex2f(bird1X, bird1Y);
        glVertex2f(bird1X + 12 + wingFlap * 3, bird1Y + 6 + wingFlap * 2);
        glVertex2f(bird1X, bird1Y);
        glVertex2f(bird1X - 12 - wingFlap * 3, bird1Y + 6 + wingFlap * 2);

        float wingFlap2 = sinf(globalTime * 0.1f + 1.0f) * 0.3f;
        glVertex2f(bird2X, bird2Y);
        glVertex2f(bird2X + 12 + wingFlap2 * 3, bird2Y + 6 + wingFlap2 * 2);
        glVertex2f(bird2X, bird2Y);
        glVertex2f(bird2X - 12 - wingFlap2 * 3, bird2Y + 6 + wingFlap2 * 2);

        float wingFlap3 = sinf(globalTime * 0.1f + 2.0f) * 0.3f;
        glVertex2f(bird3X, bird3Y);
        glVertex2f(bird3X + 12 + wingFlap3 * 3, bird3Y + 6 + wingFlap3 * 2);
        glVertex2f(bird3X, bird3Y);
```

```
        glVertex2f(bird3X - 12 - wingFlap3 * 3, bird3Y + 6 + wingFlap3 * 2);
        glEnd();
    } else {
        glBegin(GL_LINES);
        glVertex2f(bird1X, bird1Y);
        glVertex2f(bird1X + 12, bird1Y + 6);
        glVertex2f(bird1X, bird1Y);
        glVertex2f(bird1X - 12, bird1Y + 6);

        glVertex2f(bird2X, bird2Y);
        glVertex2f(bird2X + 12, bird2Y + 6);
        glVertex2f(bird2X, bird2Y);
        glVertex2f(bird2X - 12, bird2Y + 6);

        glVertex2f(bird3X, bird3Y);
        glVertex2f(bird3X + 12, bird3Y + 6);
        glVertex2f(bird3X, bird3Y);
        glVertex2f(bird3X - 12, bird3Y + 6);
        glEnd();
    }
}

void drawWaterAndWaves() {
    if (enhancedMode) {
        glBegin(GL_QUADS);
```

```
if (isDay) {
    setColor(0.05f, 0.3f, 0.6f);
    glVertex2f(0, 0);
    glVertex2f(WINDOW_WIDTH, 0);
    setColor(0.2f, 0.5f, 0.8f);
    glVertex2f(WINDOW_WIDTH, 260);
    glVertex2f(0, 260);
} else {
    setColor(0.01f, 0.01f, 0.05f);
    glVertex2f(0, 0);
    glVertex2f(WINDOW_WIDTH, 0);
    setColor(0.02f, 0.02f, 0.1f);
    glVertex2f(WINDOW_WIDTH, 260);
    glVertex2f(0, 260);
}
glEnd();

for (int layer = 0; layer < 3; ++layer) {
    if (isDay) {
        setColor(0.3f + layer * 0.1f, 0.6f + layer * 0.1f, 0.9f, 0.6f - layer * 0.15f);
    } else {
        setColor(0.05f, 0.05f, 0.15f + layer * 0.03f, 0.7f - layer * 0.2f);
    }

    glBegin(GL_QUAD_STRIP);
```

```
    for (int x = 0; x <= WINDOW_WIDTH; x += 10) {
        float waveHeight = 200 + layer * 15;
        float amplitude = 6 - layer;
        float frequency = 0.02f + layer * 0.005f;
        float phase = waveTime * (0.03f + layer * 0.01f);

        float y1 = waveHeight + amplitude * sinf(x * frequency + phase);
        float y2 = y1 + 2 + layer;

        glVertex2f(x, y1);
        glVertex2f(x, y2);
    }
    glEnd();
}

if (!isDay && sunX >= 0 && sunX <= WINDOW_WIDTH) {
    setColor(0.7f, 0.7f, 0.9f, 0.4f);
    float reflectionY = 150;

    glBegin(GL_QUAD_STRIP);
    for (int x = sunX - 40; x <= sunX + 40; x += 5) {
        if (x >= 0 && x <= WINDOW_WIDTH) {
            float distortion = sinf(x * 0.1f + waveTime * 0.1f) * 8;
            float y1 = reflectionY + distortion;
            float y2 = y1 + 25;
```

```
                float fade = 1.0f - abs(x - sunX) / 40.0f;
                setColor(0.7f * fade, 0.7f * fade, 0.9f * fade, 0.4f * fade);

                glVertex2f(x, y1);
                glVertex2f(x, y2);
            }
        }
        glEnd();
    }

} else {
    if (isDay)
        setColor(0.10f, 0.45f, 0.80f);
    else
        setColor(0.02f, 0.02f, 0.1f);

    glBegin(GL_QUADS);
    glVertex2f(0, 0);
    glVertex2f(WINDOW_WIDTH, 0);
    glVertex2f(WINDOW_WIDTH, 260);
    glVertex2f(0, 260);
    glEnd();

    if (isDay)
```

```
            setColor(0.15f, 0.55f, 0.9f);
        else
            setColor(0.04f, 0.04f, 0.15f);

        glBegin(GL_QUADS);
        for (int x = 0; x < WINDOW_WIDTH; x += 20) {
            float y = 200 + 8 * sinf(x * 0.03f + waveTime * 0.05f);
            glVertex2f(x, y);
            glVertex2f(x + 20, y);
            glVertex2f(x + 20, y + 3);
            glVertex2f(x, y + 3);
        }
        glEnd();
    }
}

void drawBoats() {
    // Fishing boat
    {
        float bobbing = 6.0f * sinf(fishingBoatX * 0.01f + waveTime * 0.07f);
        float y = 150 + bobbing;

        glPushMatrix();
        glTranslatef(fishingBoatX, y, 0);
```

```
setColor(0.55f, 0.27f, 0.07f);
glBegin(GL_POLYGON);
glVertex2f(-70, 0);
glVertex2f(70, 0);
glVertex2f(45, -20);
glVertex2f(-45, -20);
glEnd();

setColor(0.85f, 0.85f, 0.9f);
glBegin(GL_QUADS);
glVertex2f(-20, 0);
glVertex2f(20, 0);
glVertex2f(20, 25);
glVertex2f(-20, 25);
glEnd();

setColor(0.2f, 0.35f, 0.6f);
glBegin(GL_QUADS);
glVertex2f(-15, 10);
glVertex2f(-5, 10);
glVertex2f(-5, 20);
glVertex2f(-15, 20);
glEnd();

glBegin(GL_QUADS);
```

```
glVertex2f(5, 10);
glVertex2f(15, 10);
glVertex2f(15, 20);
glVertex2f(5, 20);
glEnd();

setColor(0.3f, 0.3f, 0.3f);
glBegin(GL_QUADS);
glVertex2f(22, 0);
glVertex2f(26, 0);
glVertex2f(26, 34);
glVertex2f(22, 34);
glEnd();

setColor(0.8f, 0.1f, 0.1f);
glBegin(GL_TRIANGLES);
glVertex2f(26, 34);
glVertex2f(60, 26);
glVertex2f(26, 22);
glEnd();

float manY = 25, manHeight = 15, headR = 5;
setColor(0.9f, 0.8f, 0.7f);

glBegin(GL_TRIANGLE_FAN);
```

```
glVertex2f(-20, manY + manHeight);
for (int s = 0; s <= 40; ++s) {
   float ang = 2.0f * 3.1415926f * s / 40;
   glVertex2f(-20 + headR * cosf(ang), manY + manHeight + headR * sinf(ang));
}
glEnd();

glBegin(GL_TRIANGLE_FAN);
glVertex2f(20, manY + manHeight);
for (int s = 0; s <= 40; ++s) {
   float ang = 2.0f * 3.1415926f * s / 40;
   glVertex2f(20 + headR * cosf(ang), manY + manHeight + headR * sinf(ang));
}
glEnd();

setColor(0.1f, 0.3f, 0.8f);
glBegin(GL_QUADS);
glVertex2f(-23, manY);
glVertex2f(-17, manY);
glVertex2f(-17, manY + manHeight);
glVertex2f(-23, manY + manHeight);
glEnd();

glBegin(GL_QUADS);
glVertex2f(17, manY);
```

```
    glVertex2f(23, manY);
    glVertex2f(23, manY + manHeight);
    glVertex2f(17, manY + manHeight);
    glEnd();

    glPopMatrix();
}

// Cargo boat
{
    float bobbing = 6.0f * sinf(cargoBoatX * 0.01f + waveTime * 0.07f);
    float y = 120 + bobbing;

    glPushMatrix();
    glTranslatef(cargoBoatX, y, 0);

    setColor(0.55f, 0.27f, 0.07f);
    glBegin(GL_POLYGON);
    glVertex2f(-70, 0);
    glVertex2f(70, 0);
    glVertex2f(45, -20);
    glVertex2f(-45, -20);
    glEnd();

    setColor(0.85f, 0.85f, 0.9f);
```

```
glBegin(GL_QUADS);
glVertex2f(-20, 0);
glVertex2f(20, 0);
glVertex2f(20, 25);
glVertex2f(-20, 25);
glEnd();

setColor(0.2f, 0.35f, 0.6f);
glBegin(GL_QUADS);
glVertex2f(-15, 10);
glVertex2f(-5, 10);
glVertex2f(-5, 20);
glVertex2f(-15, 20);
glEnd();

glBegin(GL_QUADS);
glVertex2f(5, 10);
glVertex2f(15, 10);
glVertex2f(15, 20);
glVertex2f(5, 20);
glEnd();

setColor(0.3f, 0.3f, 0.3f);
glBegin(GL_QUADS);
glVertex2f(22, 0);
```

```
glVertex2f(26, 0);
glVertex2f(26, 34);
glVertex2f(22, 34);
glEnd();

setColor(0.8f, 0.1f, 0.1f);
glBegin(GL_TRIANGLES);
glVertex2f(26, 34);
glVertex2f(60, 26);
glVertex2f(26, 22);
glEnd();

float manY = 25, manHeight = 15, headR = 5;
setColor(0.9f, 0.8f, 0.7f);

glBegin(GL_TRIANGLE_FAN);
glVertex2f(-20, manY + manHeight);
for (int s = 0; s <= 40; ++s) {
   float ang = 2.0f * 3.1415926f * s / 40;
   glVertex2f(-20 + headR * cosf(ang), manY + manHeight + headR * sinf(ang));
}
glEnd();

glBegin(GL_TRIANGLE_FAN);
glVertex2f(20, manY + manHeight);
```

```
        for (int s = 0; s <= 40; ++s) {
            float ang = 2.0f * 3.1415926f * s / 40;
            glVertex2f(20 + headR * cosf(ang), manY + manHeight + headR * sinf(ang));
        }
        glEnd();

        setColor(0.1f, 0.3f, 0.8f);
        glBegin(GL_QUADS);
        glVertex2f(-23, manY);
        glVertex2f(-17, manY);
        glVertex2f(-17, manY + manHeight);
        glVertex2f(-23, manY + manHeight);
        glEnd();

        glBegin(GL_QUADS);
        glVertex2f(17, manY);
        glVertex2f(23, manY);
        glVertex2f(23, manY + manHeight);
        glVertex2f(17, manY + manHeight);
        glEnd();

        glPopMatrix();
    }
}
```

```
void drawCarsAndAirplane() {
    float wheels[2] = {25, 75};
    int wheelSegs = 20;

    // red car
    glPushMatrix();
    glTranslatef(redCarX, 360, 0);
    setColor(0.9f, 0.2f, 0.2f);

    glBegin(GL_QUADS);
    glVertex2f(0, 0);
    glVertex2f(100, 0);
    glVertex2f(100, 24);
    glVertex2f(0, 24);
    glEnd();

    glBegin(GL_QUADS);
    glVertex2f(20, 24);
    glVertex2f(80, 24);
    glVertex2f(80, 50);
    glVertex2f(20, 50);
    glEnd();

    setColor(0, 0, 0);
    for (int i = 0; i < 2; ++i) {
```

```
    float cx = wheels[i], cy = 0;
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(cx, cy);
    for (int s = 0; s <= wheelSegs; ++s) {
        float ang = 2.0f * 3.1415926f * s / wheelSegs;
        glVertex2f(cx + 10 * cosf(ang), cy + 10 * sinf(ang));
    }
    glEnd();
}

setColor(0.9f, 0.95f, 1.0f);
glBegin(GL_QUADS);
glVertex2f(28, 32);
glVertex2f(52, 32);
glVertex2f(52, 48);
glVertex2f(28, 48);
glEnd();

glBegin(GL_QUADS);
glVertex2f(54, 32);
glVertex2f(76, 32);
glVertex2f(76, 48);
glVertex2f(54, 48);
glEnd();
glPopMatrix();
```

```
// green car
glPushMatrix();
glTranslatef(greenCarX, 365, 0);
glScalef(0.9f, 0.9f, 1.0f);
setColor(0.2f, 0.8f, 0.25f);

glBegin(GL_QUADS);
glVertex2f(0, 0);
glVertex2f(100, 0);
glVertex2f(100, 24);
glVertex2f(0, 24);
glEnd();

glBegin(GL_QUADS);
glVertex2f(20, 24);
glVertex2f(80, 24);
glVertex2f(80, 50);
glVertex2f(20, 50);
glEnd();

setColor(0, 0, 0);
for (int i = 0; i < 2; ++i) {
    float cx = wheels[i], cy = 0;
    glBegin(GL_TRIANGLE_FAN);
```

```
    glVertex2f(cx, cy);
    for (int s = 0; s <= wheelSegs; ++s) {
        float ang = 2.0f * 3.1415926f * s / wheelSegs;
        glVertex2f(cx + 10 * cosf(ang), cy + 10 * sinf(ang));
    }
    glEnd();
}

setColor(0.9f, 0.95f, 1.0f);
glBegin(GL_QUADS);
glVertex2f(28, 32);
glVertex2f(52, 32);
glVertex2f(52, 48);
glVertex2f(28, 48);
glEnd();

glBegin(GL_QUADS);
glVertex2f(54, 32);
glVertex2f(76, 32);
glVertex2f(76, 48);
glVertex2f(54, 48);
glEnd();
glPopMatrix();

// blue car
```

```
glPushMatrix();
glTranslatef(blueCarX, 358, 0);
glScalef(1.1f, 1.1f, 1.0f);
setColor(0.25f, 0.5f, 0.95f);

glBegin(GL_QUADS);
glVertex2f(0, 0);
glVertex2f(100, 0);
glVertex2f(100, 24);
glVertex2f(0, 24);
glEnd();

glBegin(GL_QUADS);
glVertex2f(20, 24);
glVertex2f(80, 24);
glVertex2f(80, 50);
glVertex2f(20, 50);
glEnd();

setColor(0, 0, 0);
for (int i = 0; i < 2; ++i) {
    float cx = wheels[i], cy = 0;
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(cx, cy);
    for (int s = 0; s <= wheelSegs; ++s) {
```

```
        float ang = 2.0f * 3.1415926f * s / wheelSegs;
        glVertex2f(cx + 10 * cosf(ang), cy + 10 * sinf(ang));
    }
    glEnd();
}

setColor(0.9f, 0.95f, 1.0f);
glBegin(GL_QUADS);
glVertex2f(28, 32);
glVertex2f(52, 32);
glVertex2f(52, 48);
glVertex2f(28, 48);
glEnd();

glBegin(GL_QUADS);
glVertex2f(54, 32);
glVertex2f(76, 32);
glVertex2f(76, 48);
glVertex2f(54, 48);
glEnd();
glPopMatrix();

// airplane
glPushMatrix();
glTranslatef(airplaneX, airplaneY, 0);
```

```
setColor(0.8f, 0.1f, 0.1f);
glBegin(GL_QUADS);
glVertex2f(0, 0);
glVertex2f(70, 0);
glVertex2f(70, 10);
glVertex2f(0, 10);
glEnd();

glBegin(GL_TRIANGLES);
glVertex2f(70, 0);
glVertex2f(85, 5);
glVertex2f(70, 10);
glEnd();

setColor(0.1f, 0.1f, 0.8f);
glBegin(GL_TRIANGLES);
glVertex2f(0, 10);
glVertex2f(10, 10);
glVertex2f(5, 20);
glEnd();

glBegin(GL_TRIANGLES);
glVertex2f(20, 10);
glVertex2f(50, 10);
```

```
    glVertex2f(35, -15);
    glEnd();

    glBegin(GL_TRIANGLES);
    glVertex2f(20, 0);
    glVertex2f(50, 0);
    glVertex2f(35, 15);
    glEnd();

    glPopMatrix();
}

void drawScenario0() {
    drawSky();
    drawSunOrMoon();
    drawClouds();
    drawBirds();
    drawWaterAndWaves();
    drawBoats();

    // Ground/Hill
    setColor(0.2f, 0.6f, 0.1f);
    glBegin(GL_QUADS);
    glVertex2f(0, 260);
    glVertex2f(WINDOW_WIDTH, 260);
```

```
glVertex2f(WINDOW_WIDTH, 320);
glVertex2f(0, 320);
glEnd();

// Road
setColor(0.4f, 0.4f, 0.4f);
glBegin(GL_QUADS);
glVertex2f(0, 260);
glVertex2f(WINDOW_WIDTH, 260);
glVertex2f(WINDOW_WIDTH, 280);
glVertex2f(0, 280);
glEnd();

// Road center line
setColor(1.0f, 1.0f, 0.8f);
for (int x = 0; x < WINDOW_WIDTH; x += 40) {
    glBegin(GL_QUADS);
    glVertex2f(x, 268);
    glVertex2f(x + 20, 268);
    glVertex2f(x + 20, 272);
    glVertex2f(x, 272);
    glEnd();
}

// House 1
```

```
float house1X = 120, house1Y = 320;

setColor(0.8f, 0.6f, 0.4f);
glBegin(GL_QUADS);
glVertex2f(house1X - 40, house1Y);
glVertex2f(house1X + 40, house1Y);
glVertex2f(house1X + 40, house1Y + 50);
glVertex2f(house1X - 40, house1Y + 50);
glEnd();

setColor(0.7f, 0.2f, 0.1f);
glBegin(GL_TRIANGLES);
glVertex2f(house1X - 50, house1Y + 50);
glVertex2f(house1X + 50, house1Y + 50);
glVertex2f(house1X, house1Y + 80);
glEnd();

setColor(0.4f, 0.2f, 0.1f);
glBegin(GL_QUADS);
glVertex2f(house1X - 10, house1Y);
glVertex2f(house1X + 10, house1Y);
glVertex2f(house1X + 10, house1Y + 30);
glVertex2f(house1X - 10, house1Y + 30);
glEnd();
```

```
if (isDay) {
    setColor(0.7f, 0.9f, 1.0f);
} else {
    setColor(1.0f, 0.9f, 0.3f);
}

glBegin(GL_QUADS);
glVertex2f(house1X - 35, house1Y + 35);
glVertex2f(house1X - 20, house1Y + 35);
glVertex2f(house1X - 20, house1Y + 45);
glVertex2f(house1X - 35, house1Y + 45);
glEnd();

glBegin(GL_QUADS);
glVertex2f(house1X + 20, house1Y + 35);
glVertex2f(house1X + 35, house1Y + 35);
glVertex2f(house1X + 35, house1Y + 45);
glVertex2f(house1X + 20, house1Y + 45);
glEnd();

// Trees
float treePosX[] = {60, 250, 450, 750, 850};
float treeY = 320;

for (int i = 0; i < 5; i++) {
```

```
        setColor(0.4f, 0.2f, 0.1f);
        glBegin(GL_QUADS);
        glVertex2f(treePosX[i] - 5, treeY);
        glVertex2f(treePosX[i] + 5, treeY);
        glVertex2f(treePosX[i] + 5, treeY + 40);
        glVertex2f(treePosX[i] - 5, treeY + 40);
        glEnd();

        setColor(0.1f, 0.7f, 0.1f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(treePosX[i], treeY + 50);
        for (int s = 0; s <= 40; ++s) {
            float ang = 2.0f * 3.1415926f * s / 40;
            glVertex2f(treePosX[i] + 25 * cosf(ang), treeY + 50 + 25 * sinf(ang));
        }
        glEnd();
    }

    drawCarsAndAirplane();
}

void drawBridgeScene() {
    drawSky();
    drawSunOrMoon();
    drawClouds();
```

```
drawBirds();
drawWaterAndWaves();
drawBoats();

setColor(0.15f, 0.17f, 0.20f);
glBegin(GL_QUADS);
glVertex2f(0, 350);
glVertex2f(WINDOW_WIDTH, 350);
glVertex2f(WINDOW_WIDTH, 390);
glVertex2f(0, 390);
glEnd();

setColor(1, 1, 0.2f);
for (int x = 40; x < WINDOW_WIDTH; x += 100) {
    glBegin(GL_QUADS);
    glVertex2f(x, 367);
    glVertex2f(x + 40, 367);
    glVertex2f(x + 40, 373);
    glVertex2f(x, 373);
    glEnd();
}

setColor(0.44f, 0.44f, 0.46f);
for (int x = 120; x <= 900; x += 200) {
    glBegin(GL_QUADS);
```

```
        glVertex2f(x - 10, 260);
        glVertex2f(x + 10, 260);
        glVertex2f(x + 10, 350);
        glVertex2f(x - 10, 350);
        glEnd();
    }

    drawCarsAndAirplane();
}

void drawIslandScene() {
    if (spaceMode) {
        drawNebulaBackground();
        drawDistantGalaxy();
    }

    drawSky();

    if (spaceMode) {
        drawPlanets();
        drawAuroras();
    }

    drawSunOrMoon();
```

```
if (!spaceMode) {
    drawClouds();
    drawBirds();
} else {
    drawSpaceDebris();
}

drawWaterAndWaves();
drawBoats();

float islandX = 200, islandY = 150;

if (spaceMode) {
    setColor(0.4f, 0.2f, 0.6f);
} else {
    setColor(0.6f, 0.4f, 0.2f);
}

glBegin(GL_TRIANGLE_FAN);
glVertex2f(islandX, islandY);
for (int s = 0; s <= 60; ++s) {
    float ang = 2.0f * 3.1415926f * s / 60;
    glVertex2f(islandX + 120 * cosf(ang), islandY + 40 * sinf(ang));
}
glEnd();
```

```
if (spaceMode) {
    setColor(0.2f, 0.8f, 0.9f);
} else {
    setColor(0.1f, 0.6f, 0.1f);
}

glBegin(GL_TRIANGLE_FAN);
glVertex2f(islandX, islandY + 20);
for (int s = 0; s <= 60; ++s) {
    float ang = 2.0f * 3.1415926f * s / 60;
    glVertex2f(islandX + 100 * cosf(ang), islandY + 25 * sinf(ang));
}
glEnd();

if (spaceMode) {
    setColor(0.8f, 0.3f, 0.9f);
} else {
    setColor(0.55f, 0.27f, 0.07f);
}

glBegin(GL_QUADS);
glVertex2f(islandX - 5, islandY + 20);
glVertex2f(islandX + 5, islandY + 20);
glVertex2f(islandX + 8, islandY + 80);
```

```
glVertex2f(islandX - 8, islandY + 80);
glEnd();

if (spaceMode) {
    setColor(0.3f, 0.9f, 0.8f);
    for (int i = 0; i < 4; ++i) {
        float angle = i * 1.57f;
        float crystalX = islandX + 25 * cosf(angle + globalTime * 0.01f);
        float crystalY = islandY + 90 + 15 * sinf(angle + globalTime * 0.01f);

        glBegin(GL_TRIANGLES);
        glVertex2f(crystalX, crystalY);
        glVertex2f(crystalX - 15, crystalY - 30);
        glVertex2f(crystalX + 15, crystalY - 30);
        glEnd();
    }

    float portalX = islandX + 60;
    float portalY = islandY + 40;

    for (int ring = 0; ring < 3; ++ring) {
        float ringRadius = 20 + ring * 8;
        float alpha = 0.8f - ring * 0.2f;

        setColor(0.9f, 0.5f, 1.0f, alpha);
```

```
            glBegin(GL_QUAD_STRIP);
            for (int s = 0; s <= 30; ++s) {
                float ang = 2.0f * 3.1415926f * s / 30 + globalTime * 0.02f + ring;
                float innerR = ringRadius - 2;
                float outerR = ringRadius + 2;

                glVertex2f(portalX + innerR * cosf(ang), portalY + innerR * sinf(ang));
                glVertex2f(portalX + outerR * cosf(ang), portalY + outerR * sinf(ang));
            }
            glEnd();
        }

        setColor(1.0f, 1.0f, 1.0f, 0.9f);
        glBegin(GL_TRIANGLE_FAN);
        glVertex2f(portalX, portalY);
        for (int s = 0; s <= 20; ++s) {
            float ang = 2.0f * 3.1415926f * s / 20;
            glVertex2f(portalX + 8 * cosf(ang), portalY + 8 * sinf(ang));
        }
        glEnd();
    } else {
        setColor(0.0f, 0.8f, 0.2f);
        glBegin(GL_TRIANGLES);
        glVertex2f(islandX, islandY + 80);
        glVertex2f(islandX - 40, islandY + 100);
```

```
        glVertex2f(islandX, islandY + 110);

        glVertex2f(islandX, islandY + 80);
        glVertex2f(islandX + 40, islandY + 100);
        glVertex2f(islandX, islandY + 110);

        glVertex2f(islandX, islandY + 80);
        glVertex2f(islandX - 20, islandY + 130);
        glVertex2f(islandX, islandY + 115);

        glVertex2f(islandX, islandY + 80);
        glVertex2f(islandX + 20, islandY + 130);
        glVertex2f(islandX, islandY + 115);
        glEnd();
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    switch (currentScenario) {
        case 0:
            drawScenario0();
            break;
        case 1:
```

```
            drawBridgeScene();
            break;
        case 2:
            drawIslandScene();
            break;
        default:
            drawScenario0();
            break;
    }

    glutSwapBuffers();
}

void update(int value) {
    globalTime += 1.0f;

    if (transitionLock > 0) --transitionLock;

    updateShootingStars();
    updateSpaceElements();

    if (currentScenario == 1) {
        cloud1X += 0.5f; if (cloud1X > 1100) cloud1X = -100;
        cloud2X += 0.3f; if (cloud2X > 1100) cloud2X = -100;
        cloud3X += 0.4f; if (cloud3X > 1100) cloud3X = -100;
```

```
    cloud4X += 0.6f; if (cloud4X > 1100) cloud4X = -100;

    bird1X += 3.2f; if (bird1X > 1100) bird1X = -50;
    bird2X += 2.5f; if (bird2X > 1100) bird2X = -50;
    bird3X += 2.8f; if (bird3X > 1100) bird3X = -50;
} else {
    cloud1X += 0.2f; if (cloud1X > 1100) cloud1X = -100;
    cloud2X += 0.15f; if (cloud2X > 1100) cloud2X = -100;
    cloud3X += 0.18f; if (cloud3X > 1100) cloud3X = -100;
    cloud4X += 0.22f; if (cloud4X > 1100) cloud4X = -100;

    bird1X += 1.6f; if (bird1X > 1100) bird1X = -50;
    bird2X += 1.25f; if (bird2X > 1100) bird2X = -50;
    bird3X += 1.4f; if (bird3X > 1100) bird3X = -50;
}

if (currentScenario == 1) {
    redCarX += redCarV; if (redCarX > 1100) redCarX = -200;
    greenCarX += greenCarV; if (greenCarX > 1100) greenCarX = -300;
    blueCarX += blueCarV; if (blueCarX > 1100) blueCarX = -250;
    airplaneX += 3.0f; if (airplaneX > 1100) airplaneX = -150;
}

fishingBoatX += fishingBoatV;
cargoBoatX += cargoBoatV;
```

```
bool shouldTransition = false;
const float rightExitMargin = WINDOW_WIDTH + 100.0f;
const float leftExitMargin = -200.0f;

if (transitionLock == 0) {
    if (fishingBoatV > 0.0f && cargoBoatV > 0.0f) {
        if (fishingBoatX > rightExitMargin && cargoBoatX > rightExitMargin)
            shouldTransition = true;
    } else if (fishingBoatV < 0.0f && cargoBoatV < 0.0f) {
        if (fishingBoatX < leftExitMargin && cargoBoatX < leftExitMargin)
            shouldTransition = true;
    }
}

if (shouldTransition) {
    currentScenario = (currentScenario + 1) % 3;
    transitionLock = 60;

    if (currentScenario == 0) {
        fishingBoatX = -300.0f; cargoBoatX = -900.0f;
        fishingBoatV = fabs(fishingBoatSpeed); cargoBoatV = fabs(cargoBoatSpeed);
    } else if (currentScenario == 1) {
        fishingBoatX = -300.0f; cargoBoatX = -900.0f;
        fishingBoatV = fabs(fishingBoatSpeed); cargoBoatV = fabs(cargoBoatSpeed);
```

```
            redCarX = -250.0f; greenCarX = -600.0f; blueCarX = -1000.0f;
            airplaneX = -200;
        } else if (currentScenario == 2) {
            fishingBoatX = WINDOW_WIDTH + 200.0f; cargoBoatX = WINDOW_WIDTH
+ 500.0f;
            fishingBoatV = -fabs(fishingBoatSpeed); cargoBoatV = -fabs(cargoBoatSpeed);
        }
    }

    if (currentScenario == 2) {
        if (fishingBoatX >= 100.0f && fishingBoatX <= 300.0f) fishingBoatV = 0.0f;
        if (cargoBoatX >= 100.0f && cargoBoatX <= 300.0f) cargoBoatV = 0.0f;
    }

    sunX += sunSpeed * cosf(sunAngle);
    sunY += sunSpeed * sinf(sunAngle);
    if (sunY > WINDOW_HEIGHT || sunX > WINDOW_WIDTH) {
        isDay = !isDay;
        sunX = 870; sunY = 560;
        if (isDay) {
            shootingStars.clear();
            shootingStarTimer = 0;
        }
    }
```

```
    waveTime += 1.0f;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0);
}

void keyboard(unsigned char key, int x, int y) {
    switch (key) {
        case 'n': case 'N':
            currentScenario = (currentScenario + 1) % 3;
            transitionLock = 30;
            break;
        case 'p': case 'P':
            currentScenario = (currentScenario + 2) % 3;
            transitionLock = 30;
            break;
        case 'w': case 'W':
            fishingBoatV = fabs(fishingBoatSpeed);
            cargoBoatV = fabs(cargoBoatSpeed);
            break;
        case 's': case 'S':
            fishingBoatV = -fabs(fishingBoatSpeed);
            cargoBoatV = -fabs(cargoBoatSpeed);
            break;
        case 'a': case 'A':
```

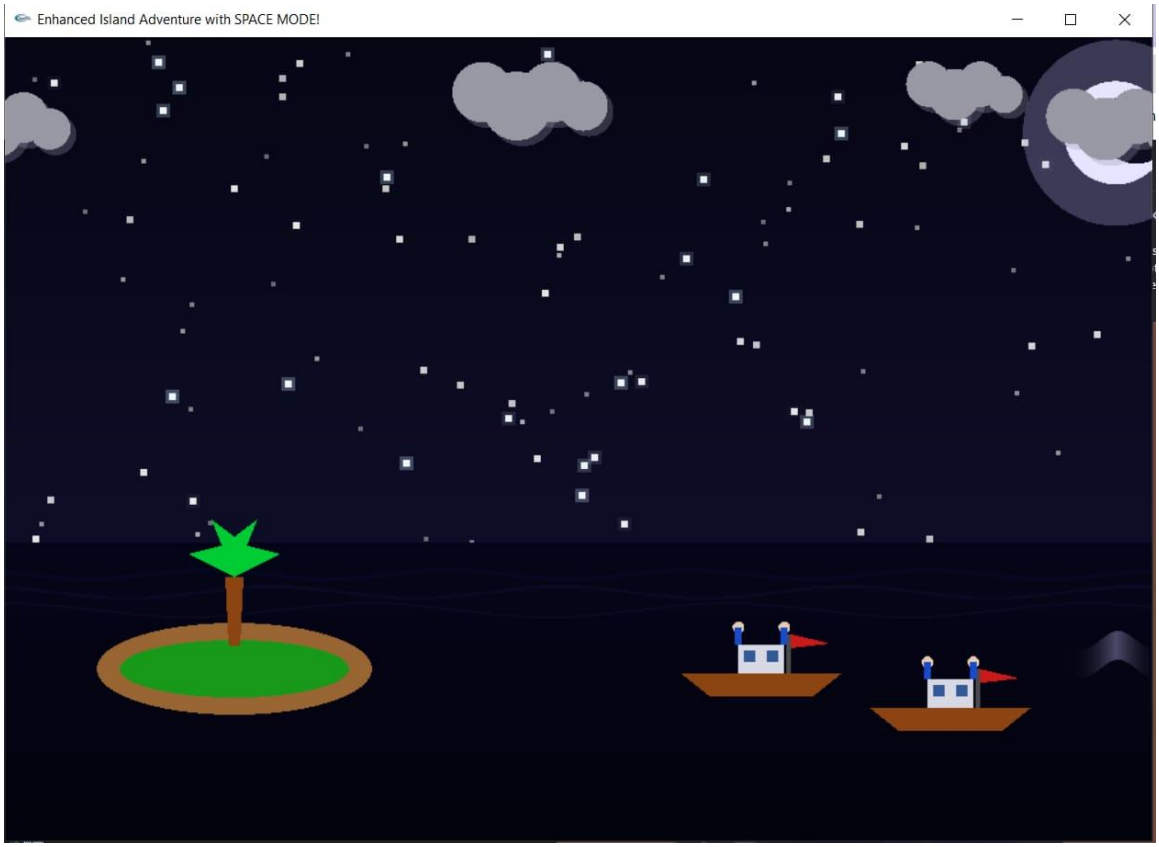
```
    fishingBoatV = 0.0f; cargoBoatV = 0.0f;
    break;
case 'f': case 'F':
    fishingBoatSpeed *= 1.5f; cargoBoatSpeed *= 1.5f;
    if (fishingBoatV > 0) fishingBoatV = fishingBoatSpeed;
    else if (fishingBoatV < 0) fishingBoatV = -fishingBoatSpeed;
    if (cargoBoatV > 0) cargoBoatV = cargoBoatSpeed;
    else if (cargoBoatV < 0) cargoBoatV = -cargoBoatSpeed;
    break;
case 'g': case 'G':
    fishingBoatSpeed *= 0.7f; cargoBoatSpeed *= 0.7f;
    if (fishingBoatV > 0) fishingBoatV = fishingBoatSpeed;
    else if (fishingBoatV < 0) fishingBoatV = -fishingBoatSpeed;
    if (cargoBoatV > 0) cargoBoatV = cargoBoatSpeed;
    else if (cargoBoatV < 0) cargoBoatV = -cargoBoatSpeed;
    break;
case 'm': case 'M':
    if (!isDay) createShootingStar();
    break;
case 'e': case 'E':
    enhancedMode = !enhancedMode;
    break;
case 'q': case 'Q':
    spaceMode = !spaceMode;
    if (spaceMode && planets.empty()) {
```

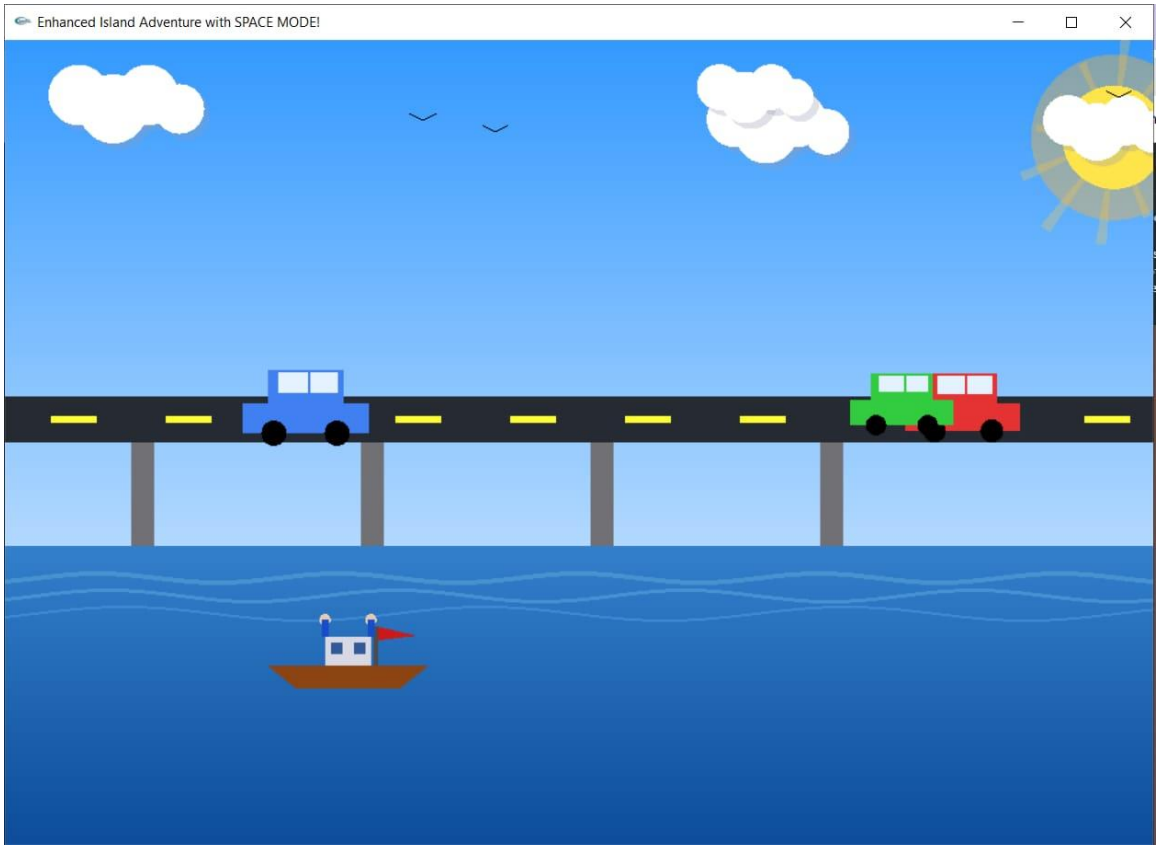
```
            initSpaceElements();
        }
        break;
    case 27:
        exit(0);
        break;
  }
  glutPostRedisplay();
}

void reshape(int w, int h) {
  glViewport(0, 0, w, h);
  glMatrixMode(GL_PROJECTION);
  glLoadIdentity();
  gluOrtho2D(0, WINDOW_WIDTH, 0, WINDOW_HEIGHT);
  glMatrixMode(GL_MODELVIEW);
  glLoadIdentity();
}

int main(int argc, char** argv) {
  srand(static_cast<unsigned int>(time(NULL)));

  glutInit(&argc, argv);
  glutInitWindowSize(WINDOW_WIDTH, WINDOW_HEIGHT);
  glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
```

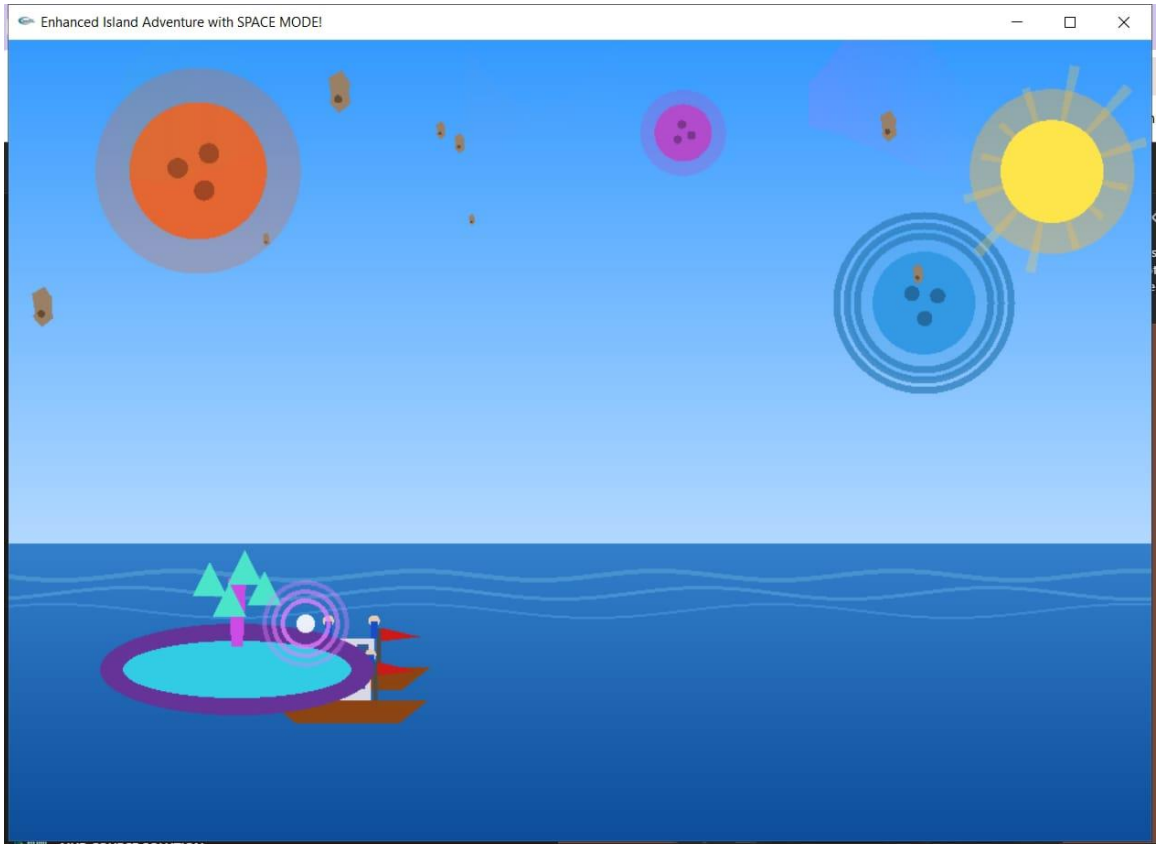
```
glutCreateWindow("Enhanced Island Adventure with SPACE MODE!");

init2D();
initStars();
initSpaceElements();

glutDisplayFunc(display);
glutReshapeFunc(reshape);
glutKeyboardFunc(keyboard);
glutTimerFunc(0, update, 0);

std::cout << "=== Enhanced Island Adventure with SPACE MODE ===\n";
std::cout << "Controls:\n";
std::cout << "Q - Toggle SPACE MODE (planets, nebula, alien landscape!)\n";
std::cout << "E - Toggle enhanced visual mode\n";
std::cout << "M - Create shooting star (night only)\n";
std::cout << "N/P - Next/Previous scenario\n";
std::cout << "W/S/A - Boat controls (forward/back/stop)\n";
std::cout << "F/G - Speed up/slow down boats\n";
std::cout << "ESC - Exit\n\n";
std::cout << "Space Mode Features:\n";
std::cout << "- Three animated planets with orbital motion\n";
std::cout << "- Nebula background with distant galaxy\n";
std::cout << "- Aurora effects and space debris\n";
std::cout << "- Alien island with crystal formations\n";
```

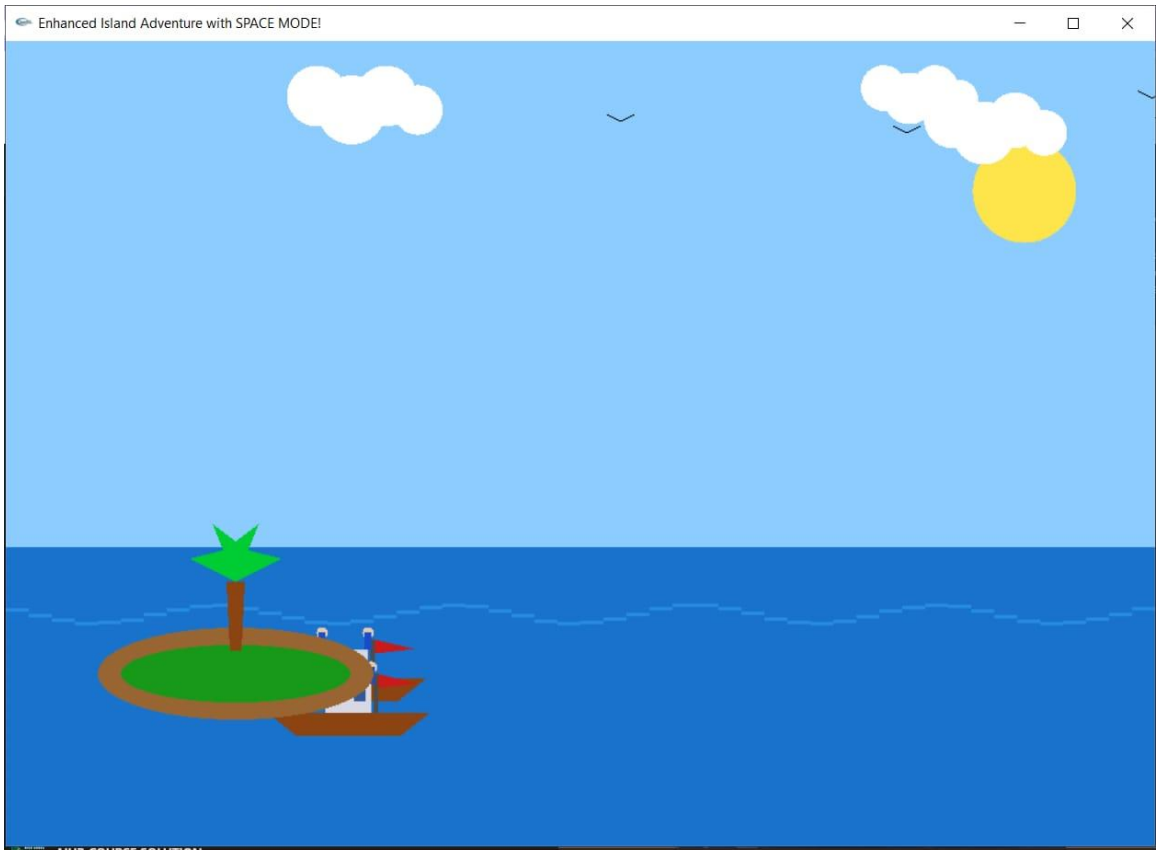
```
    std::cout << "- Mystical portal with rotating rings\n\n";
    std::cout << "Enjoy your space adventure!\n";

    glutMainLoop();
    return 0;
}
```

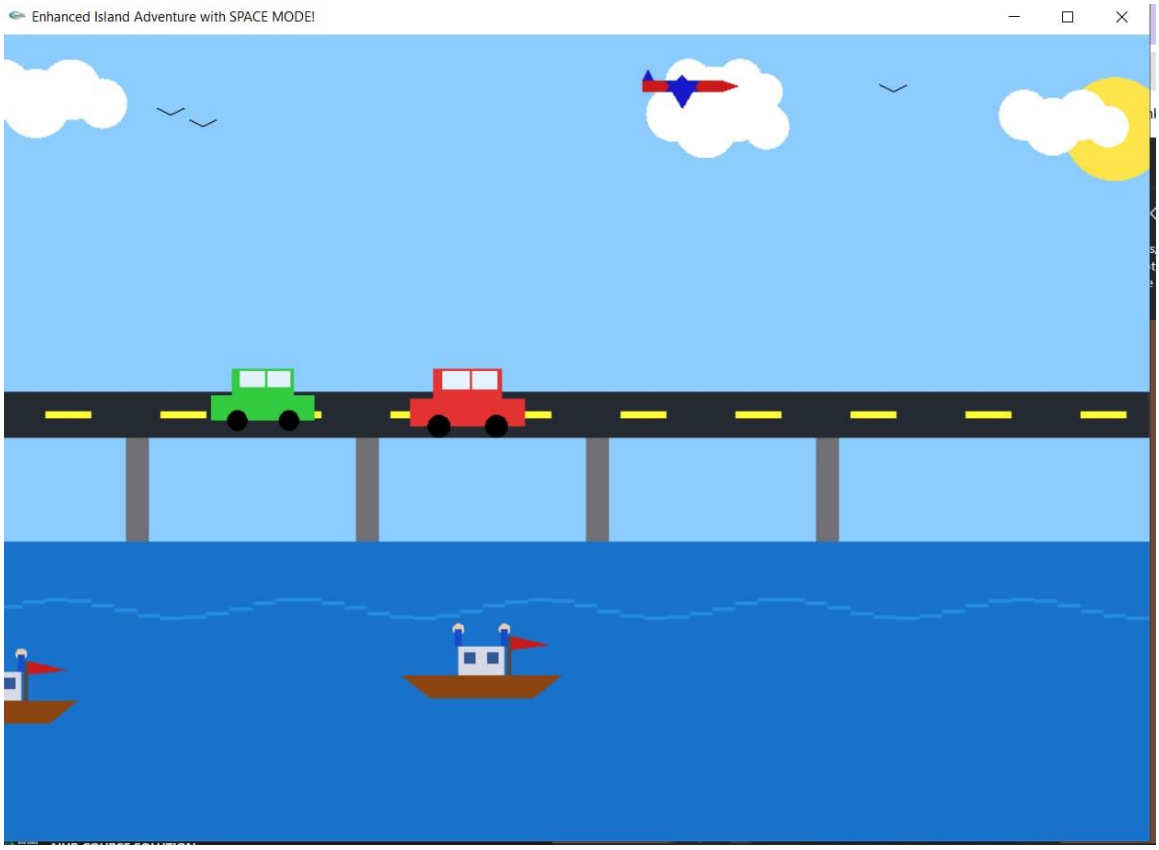
Screenshots of the Project:

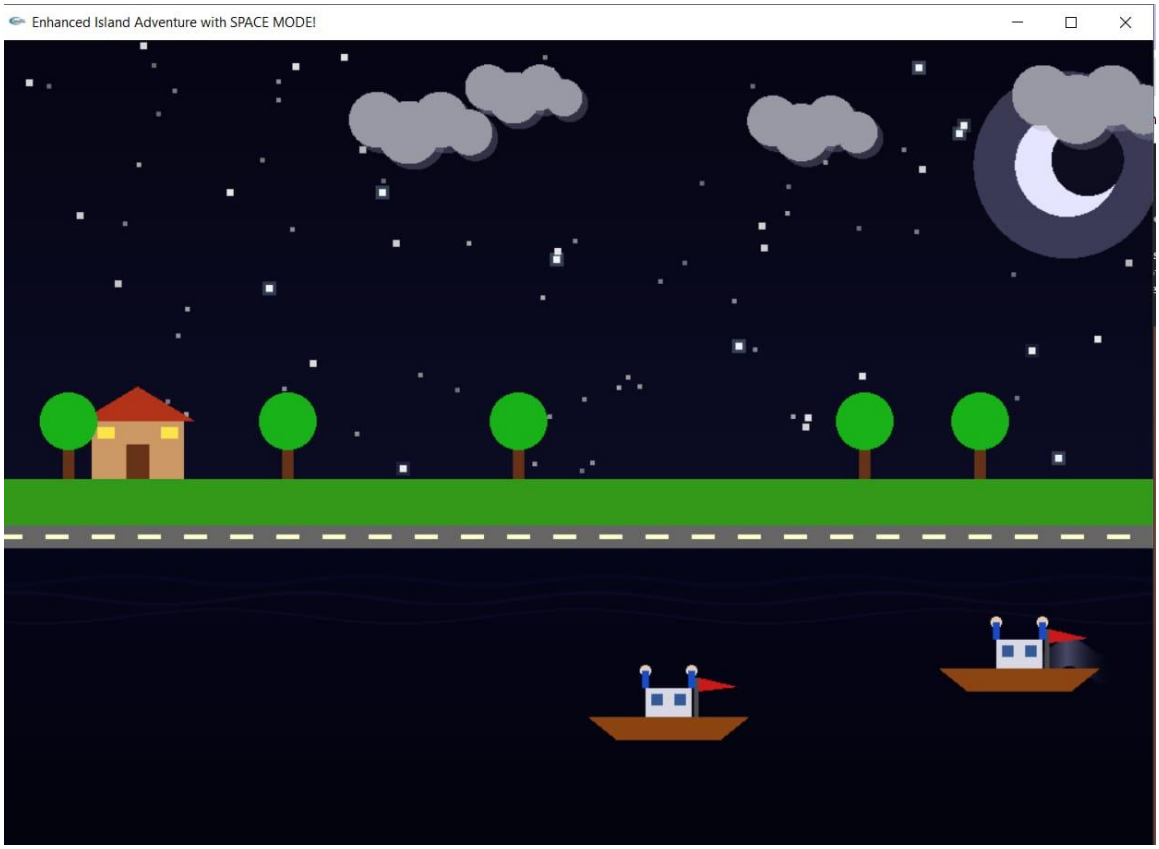


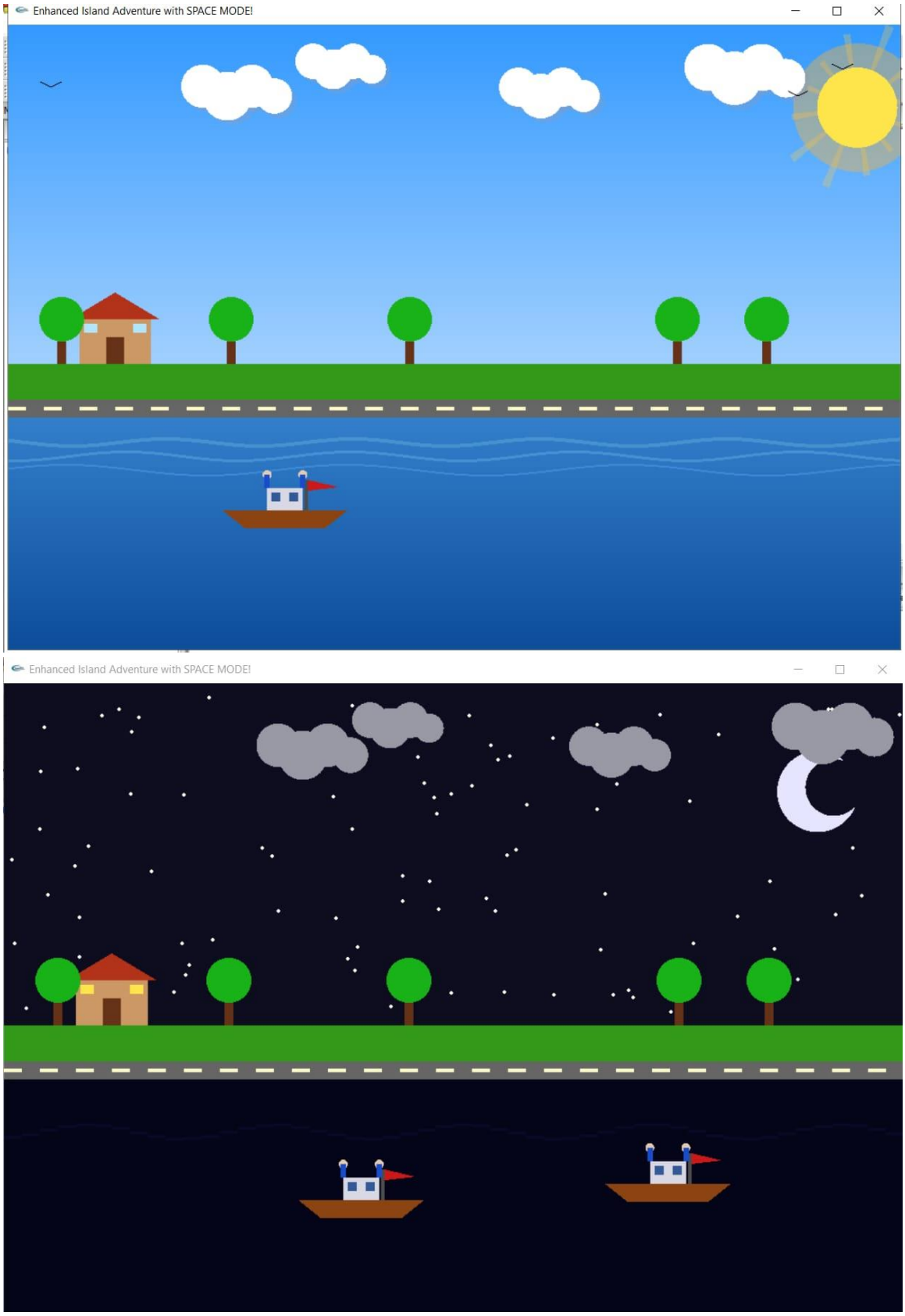


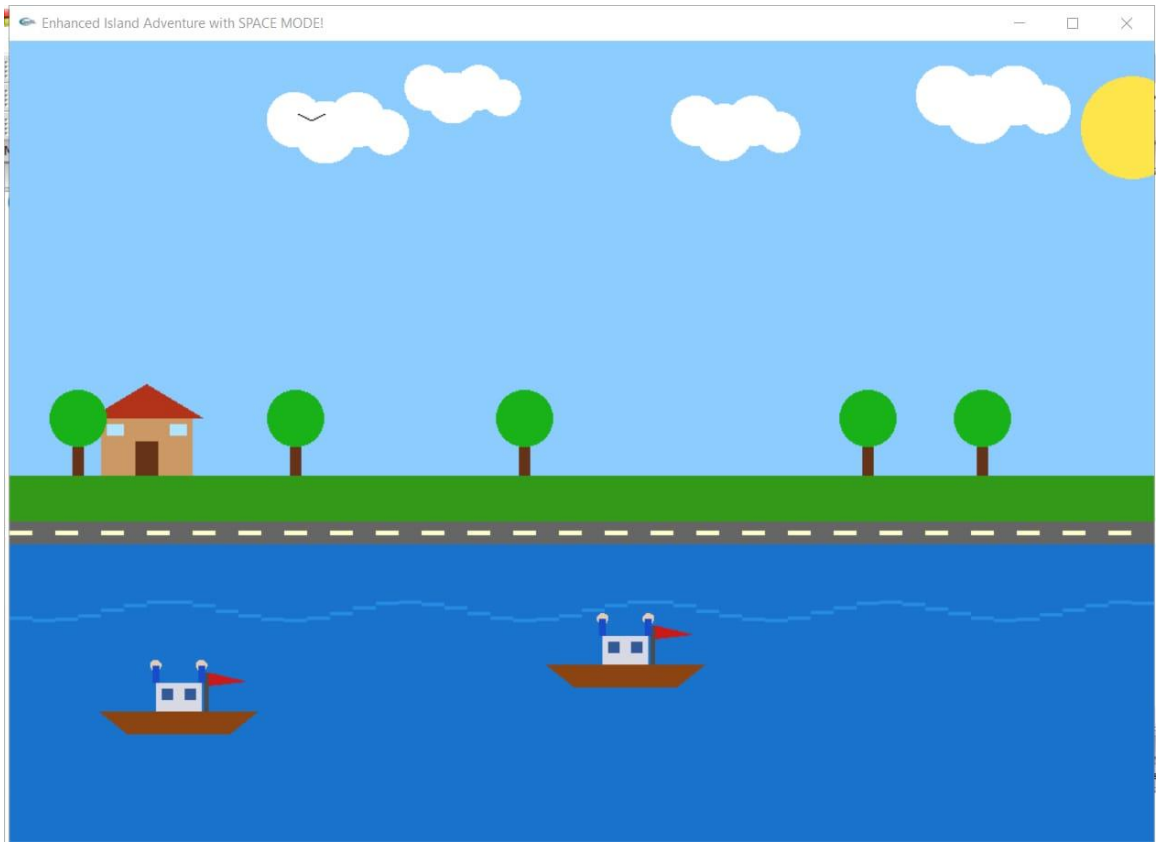

Enhanced Island Adventure with SPACE MODE!

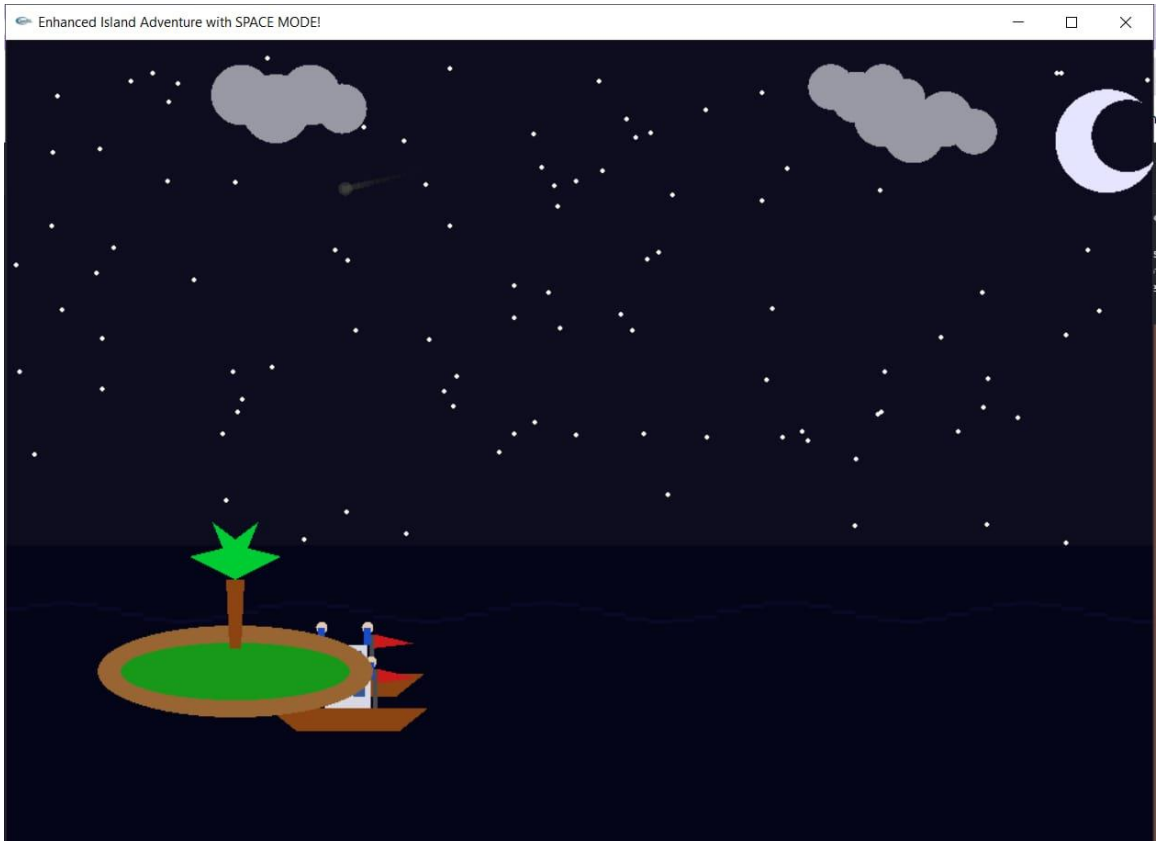

Enhanced Island Adventure with SPACE MODE!


Enhanced Island Adventure with SPACE MODE!


Enhanced Island Adventure with SPACE MODE!


Enhanced Island Adventure with SPACE MODE!
Enhanced Island Adventure with SPACE MODE!


Enhanced Island Adventure with SPACE MODE!


Enhanced Island Adventure with SPACE MODE!

Link of the implementation in GitHub:
https://github.com/Pretom58/Finding-Island-

List of References

1. OpenGL Programming Guide (The Red Book).
2. Angel, E., & Shreiner, D. Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL.
3. Khronos Group – OpenGL Official Documentation.
4. Foley, van Dam, Feiner, Hughes – Computer Graphics: Principles and Practice.
5. TutorialsPoint – OpenGL Basics.
6. LearnOpenGL.com – Modern OpenGL tutorials and examples.
7. Project experience and implementation notes.